

转债蒙特卡洛定价的改进及应用：测度变换与张量计算

核心观点：

- **可转债定价依赖于正股股价走势，可采用蒙特卡洛模拟对转债进行定价：**可转债同时具有债性和股性的特征，转债的价值依赖于未来完整的股价路径，即内嵌路径依赖型期权。我们对正股价格与转债条款的假设如下：1) 正股价格的对数过程服从几何布朗运动；2) 正股价格在到期收益率测度下服从鞅过程；3) 转债仅可能下修或赎回，不考虑回售条款；4) 在转股期内若提前转股更有利，则立刻转股。蒙特卡洛模拟能够根据指定的随机过程，模拟正股在未来一段时间内的完整价格走势、条款触发情况与执行情况，解决路径依赖问题。
- **重要性抽样、最小二乘蒙特卡洛与 Tensor 数据结构等改进蒙特卡洛模型：**原有蒙特卡洛模拟具有模拟效率较低、无法识别期权最优停时点、未考虑赎回/下修最新公告等问题，我们对蒙特卡洛模型进行了以下改进：1) 采用重要性抽样法将真实测度变换至到期收益率测度，同时可实现提高模拟效率、降低定价误差的效果；2) 采用最小二乘蒙特卡洛模拟估计转债的存续价值，确定投资者的最佳转股时点；3) 采用 Tensor 数据结构，支持 GPU 计算与自动求导，提高代码运行效率；4) 将赎回/下修最新公告处理为是否处于赎回/下修限制期、限制期时长等新参数，加入到原有蒙特卡洛模型中。
- **蒙特卡洛定价模型参数分析：**蒙特卡洛模型中大部分参数可从市场或转债募集说明书/公告中获取，但正股收益率、正股波动率、下修概率与赎回概率 4 个参数无法直接获取，需要进行合理估计。根据测算结果，我们选择最近 6 月年化收益率的 21 天滚动均值、最近 6 月年化波动率作为正股收益率与波动率的取值，赎回/下修概率则依据正股价格/转股价格的比值，在 0-0.3 之间分段取值；另外，为兼顾定价准确与计算效率，我们在定价时取模拟次数为 2500 次。同时，我们在时间序列和截面上均对模型定价的准确性进行了验证，定价结果表明平均定价误差在合理范围内，改进后的蒙特卡洛模型定价准确性较高。
- **定价方法在转债组合构建中应用广泛，定价误差可改进“双低”转债策略获取超额收益：**截至 5 月 24 日，转债市场现存余额 7908.88 亿元，共计 535 只转债；近期转债市场与 A 股市场同步回暖，转债成交金额和平均换手均呈现上升趋势，为构建转债配置策略提供了市场基础。蒙特卡洛定价方法在转债组合构建中具有广泛应用，以双低策略为例，定价误差反映了市场预期收益与历史收益的分歧，依据定价误差可找到市场给予乐观预期的转债构建配置策略，从 2023 年 6 月 30 日至 2024 年 4 月 30 日，等权加权策略相对基准中证转债指数实现超额年化收益 11.93%，波动率倒数加权策略实现超额年化收益 7.94%，最大回撤均有所收窄，表明策略可获得稳健的超额收益。
- **风险提示：**报告结论基于历史价格信息和统计规律，但二级市场受各种即时性政策影响易出现统计规律之外的走势，所以报告结论有可能无法正确预测市场发展，报告阅读者需审慎参考报告结论。证券历史收益不代表未来业绩表现，文中观点仅供参考，不构成投资建议。

—可转债定价模型系列研究

分析师

马普凡

☎：021-68597610

✉：mapufan_yj@chinastock.com.cn

分析师登记编码：S0130522040002

研究助理：刘璐

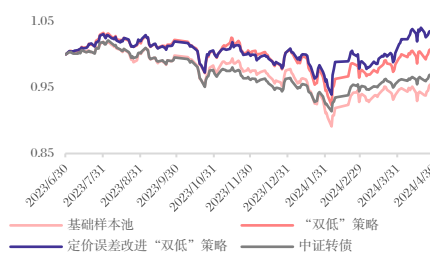
✉：liulu_yj@chinastock.com.cn

等权加权定价误差改进“双低”策略表现



资料来源：Wind，中国银河证券研究院

波动率倒数加权定价误差改进“双低”策略表现



资料来源：Wind，中国银河证券研究院

相关研究

【银河金工】转债新规下的定价模型更新和绝对收益策略改进

【银河金工】转债在“固收+”产品中的重要作用

目录

一、转债蒙特卡洛定价模型梳理与完善	3
(一) 转债定价基本假设	3
(二) 蒙特卡洛模拟适用于可转债定价	5
(三) 转债蒙特卡洛定价模型的改进：重要性抽样	6
(四) 转债蒙特卡洛定价模型的改进：最小二乘蒙特卡洛模拟	8
(五) 转债蒙特卡洛定价模型的改进：Tensor 数据结构	9
(六) 转债定价模型参数释义	10
(七) 正股价格路径模拟与转债定价流程	11
二、转债蒙特卡洛定价模型参数分析	12
(一) 定价模型参数取值方法	12
(二) 模型定价准确性验证	16
(三) 定价影响因素敏感性分析	17
三、转债市场变化	22
(一) 转债市场整体环境	22
(二) 转债行业分布与变化	23
四、基于蒙特卡洛定价模型的转债配置策略	24
(一) 定价误差改进“双低”转债策略	24
(二) 回测结果	25
五、结论与改进	27
六、附录	28
定价模型代码实例	28
七、风险提示	32

一、转债蒙特卡洛定价模型梳理与完善

（一）转债定价基本假设

在使用蒙特卡洛模拟法对转债进行定价前，我们首先需要对转债及对应正股的特征进行定义，本报告对于正股价格与转债条款的基本假设如下：

（1）正股价格的对数过程服从几何布朗运动（Geometric Brownian motion, GBM）。

股票价格服从几何布朗运动是随机过程中常用的基本假设，因为一方面经验事实证明股价的连续复利收益率近似服从正态分布，另一方面几何布朗运动是一个马尔科夫过程，即当前股价包含了已知的全部信息，这与弱有效市场假说相符。用数学公式可表示为：

$$\frac{dS_t}{S_t} = (\mu - q)dt + \sigma dW_t$$

其中 μ 为漂移率，在真实测度下 μ 为正股期望收益率，在风险中性测度下 μ 为无风险利率 r_f ； q 为正股的连续红利率， σ 为正股对数收益率的波动率， W_t 服从标准布朗运动，即 $dW_t \sim N(0, dt)$ 。考虑到短期内分红对转债价格影响较小，且分红后转股价格会相应调整，为简化模型，暂不考虑分红，即 $q = 0$ ，则正股价格可表示为

$$\frac{dS_t}{S_t} = \mu dt + \sigma dW_t$$

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

根据伊藤引理可知，若 f 是一个关于股价 S 和时间 t 的函数 $f(S, t)$ ，那么伊藤过程可表示为

$$df = \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S} dS + \frac{1}{2} \frac{\partial^2 f}{\partial S^2} (dS)^2$$

其中 $(dS)^2 = (\mu S dt + \sigma S dW)^2$ ，忽略所有比 dt 更高阶的小量，则 $(dS)^2 = (\sigma S dW)^2 = \sigma^2 S^2 dt$ ；同时将 $dS = \mu S dt + \sigma S dW$ 带入上式，可得

$$df = \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S} (\mu S dt + \sigma S dW) + \frac{1}{2} \frac{\partial^2 f}{\partial S^2} \sigma^2 S^2 dt = \left(\frac{\partial f}{\partial t} + \frac{\partial f}{\partial S} \mu S + \frac{1}{2} \frac{\partial^2 f}{\partial S^2} \sigma^2 S^2 \right) dt + \frac{\partial f}{\partial S} \sigma S dW$$

令 $f = \ln S$ ，则 $\frac{\partial f}{\partial t} = 0$ ， $\frac{\partial f}{\partial S} = \frac{1}{S}$ ， $\frac{\partial^2 f}{\partial S^2} = -\frac{1}{S^2}$ ，带入上式可得

$$d \ln S = \ln \left(\frac{S_{t+1}}{S_t} \right) = \left(0 + \frac{1}{S} \mu S - \frac{1}{2} \frac{1}{S^2} \sigma^2 S^2 \right) dt + \frac{1}{S} \sigma S dW = \left(\mu - \frac{1}{2} \sigma^2 \right) dt + \sigma dW$$

因此股票连续复利收益率服从正态分布 $d \ln S \sim N \left(\left(\mu - \frac{1}{2} \sigma^2 \right) dt, \sigma^2 dt \right)$ ，股票价格的解析式为

$$S_t = S_0 \times \exp \left[\left(\mu - \frac{\sigma^2}{2} \right) t + \sigma W_t \right]$$

（2）正股价格在到期收益率测度下服从鞅过程。

资产定价的本质是对未来现金流贴现计算现值的期望。但真实世界中，在对衍生品定价时，由于标的资产价格收益率 μ 与适合的贴现率 r 均难以进行准确估计，我们引入了测度变换（Change of Measure）的概念。测度变换可理解为一个变换概率分布的过程，是一种用于简化衍生品定价的数学工具，本身并不具有经济学含义。假定真实世界中的概率分布为真实测度，记作 P 测度；变换后的目标概率测度为 Q 测度，如

果 P 测度和 Q 测度满足二者对不可能事件的概率都为 0、对必然事件的概率都为 1，对其他事件集合可能赋予不同的概率密度，那么二者可称为等价测度。在衍生品定价中，等价测度意味着资产价格路径是一致的，但发生的概率不同。这样，只要将 Q 测度下衍生品期望价格变换回 P 测度中，我们就可以完成对真实世界中衍生品的定价。

鞅过程（Martingale）是随机过程的一种特殊形式。假设 X 为一个随机过程（即随时间变化的随机变量）， $X(t)$ 为该随机过程当前值， $\mathcal{F}_t$ 为 t 时刻所有已知信息， $X(s)$ 为该随机过程的未来值（ $s > t$ ），若 X 满足

$$E_t[X(s)|\mathcal{F}_t] = X(t)$$

则随机过程 X 被称为一个鞅过程。鞅过程在金融建模中被广泛应用，这来源于有效市场的假设：如果市场有效，未来资产价格的期望等于当前价格，那么任何一种资产平均而言都无法获得超额收益。如果我们找到一个和 P 测度等价的 Q 测度，使得资产价格序列在 Q 测度下是一个鞅过程，这样的 Q 测度就称为等价鞅测度。在无套利的假设下，等价鞅测度（Q 测度）与真实测度（P 测度）的定价具有统一性。

至此可见，对可转债的定价关键在于找到一个合适的等价鞅测度 Q。实际上，我们可以证明如果正股价序列记作 S_t ，那么存在一个等价鞅测度 Q 使得股价的贴现过程 $S_t^* = S_t e^{-rt}$ 是一个鞅，其中 r 是贴现率。在前文中，我们已建立了在真实测度（P 测度）下正股价满足几何布朗运动的假设，即

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

由于存在漂移项 μ ，股票价格存在长期趋势，显然不满足鞅过程的定义。令股价贴现过程 $S_t^* = S_t e^{-rt}$ ，则有 $\frac{\partial S_t^*}{\partial t} = -r e^{-rt} S_t = -r S_t^*$ ， $\frac{\partial S_t^*}{\partial S_t} = e^{-rt} = \frac{S_t^*}{S_t}$ ， $\frac{\partial^2 S_t^*}{\partial S_t^2} = 0$ ，根据伊藤引理可得

$$\begin{aligned} dS_t^* &= \left(\frac{\partial S_t^*}{\partial t} + \frac{\partial S_t^*}{\partial S_t} \mu S_t + \frac{1}{2} \frac{\partial^2 S_t^*}{\partial S_t^2} \sigma^2 S_t^2 \right) dt + \frac{\partial S_t^*}{\partial S_t} \sigma S_t dW_t = \left(-r S_t^* + \mu \frac{S_t^*}{S_t} S_t \right) dt + \sigma \frac{S_t^*}{S_t} S_t dW_t \\ &= (\mu - r) S_t^* dt + \sigma S_t^* dW_t \end{aligned}$$

在 P 测度下 μ 是股票长期收益率，与贴现率 r 不一定相等，因此股价贴现过程 S_t^* 仍然不是一个鞅过程，因此需要进行测度变换。引入单位波动率的风险溢价 $\theta = (\mu - r)/\sigma$ ，可得

$$dS_t^* = \frac{\mu - r}{\sigma} \sigma S_t^* dt + \sigma S_t^* dW_t = \sigma S_t^* (\theta dt + dW_t)$$

令 $dW_t^* = \theta dt + dW_t$ ，则有 $dS_t^* = \sigma S_t^* dW_t^*$ ，此时随机过程 S_t^* 中仅含有 dW_t^* 项，满足鞅过程的定义。实际上， $dW_t^* = \theta dt + dW_t$ 对布朗运动 dW_t 进行平移即是完成了测度变换，根据 Girsanov 定理在新测度 Q 下 dW_t^* 仍是一个布朗运动，且 Q 测度与 P 测度等价。在 Q 测度中股价贴现序列不再含有漂移项 μ ，

$$\frac{dS_t^*}{S_t^*} = \sigma dW_t^*$$

因此在 Q 测度中 S_t^* 满足 $S_t^* = E_t[S_T^*]$ ， $T > t$ ，即在 Q 测度中 $S_t^* = S_t e^{-rt}$ 是一个鞅过程。而贴现因子 e^{-rt} 也可视为一种计价单位，定义 $R_t = e^{rt}$ ， r 为连续复利收益率。若计价单位为货币市场账户、收益率为无风险利率 r_f ，则变换后的 Q 测度称为风险中性测度，这是衍生品定价中最常用的测度。

测度变换有效的来源是风险的可对冲性。以欧式股票看涨期权为例，假设我们在卖出一单位看涨期权 C 的同时买入 Delta 单位标的股票 S，则股价 S 变动的风险就实现了完全对冲，组合的期望收益仅来源于卖出期权得到的现金流 C 产生的收益，即对应无风险利率 r_f 。

但对于可转债而言，风险中性测度并不适用。即使可转债转股的风险可以被完全对冲，转债的票息等现金流仍然存在信用风险；而且票息等现金流也会对转债价值产生影响，即使当前股价高于转股价，投资者也需要评估立刻转股的收益与继续持有转债获取票息的收益以决定是否行权。因此，**风险中性测度并不**

适合转债定价，我们应当使用转债的到期收益率进行贴现以反映转债现金流的信用风险。为便于表达，后文中我们统一将贴现率为转债到期收益率的测度称为到期收益率测度。

（3） 转债仅可能发生赎回或下修，不考虑回售条款。

在正股表现不佳时，股价首先触发下修条款，企业为防止回售，可能会选择下修转股价，尤其在回售期临近的时期。若发生回售，则发行方转债融资失败，这与大多数上市公司的目的相悖。假定在**触发下修/赎回条款后，发行方需要一定时间选择是否执行下修/赎回**，因为在触发条款后往往会经过董事会提议/审议，股东大会审议与保荐公司核查等程序。此外，我们还假定**上市公司在决定不下修/不赎回后，转债会进入“限制”状态**：在一段时间内，若再触发条款则仍不下修/不赎回，且触发条件从非限制期开始计算。

（4） 投资者是理性的，在转股期内如果提前转股更有利，则立刻转股。

在任何可以进行转股的时间点，如果转债的内在价值（立刻转股的价值）大于转债存续价值（继续持有转债的价值）的期望，则投资者会立刻转股。

（二）蒙特卡洛模拟适用于可转债定价

基于前文的基本假设，在为可转债定价时，需要考虑：1）投资者在持有转债期间可能享有的票息与面值；2）进入转股期后，投资者相当于享有以正股为标的物美式看涨期权；3）转债发行方享有转股价格的修正权与转债赎回权。其中，可转债条款的触发情况直接与未来的股价路径相关，这将改变未来的现金流，进而影响转债的定价。具体而言：

- 如果正股未来的股价在某段时间内维持高位，可能触发赎回条款。若此时上市公司选择赎回转债，那么投资者在转债赎回后将无法获得票息；若上市公司不赎回，那么在未来一段时间内定价时不再考虑赎回条款。
- 如果正股未来的股价在某短时间内处于低位，可能触发下修条款。若此时上市公司选择下修转股价，将有利于投资者尽快执行转股，获得转股价值；若上市公司不下修，那么在未来一段时间内定价时不再考虑下修条款。

因此，转债的价值依赖于未来完整的股价路径，即内嵌路径依赖型期权。蒙特卡洛模拟（Monte Carlo Simulation）能够根据指定的随机过程，模拟正股在未来一段时间内的完整价格走势、条款触发情况与执行情况，解决路径依赖问题。蒙特卡洛模拟本身并不复杂，容易实现。此外，蒙特卡洛模拟还具有较强的扩展性，可根据不同的市场情景，添加新的变量或改变变量的设定，不必对原有程序做大幅修改。

在前期报告《转债新规下的定价模型更新和绝对收益策略改进》中，我们将条款新规写入定价模型，使模拟触发条款和执行条款的过程更符合市场真实情况，但该报告的定价程序存在以下问题：

- （1） 在原有蒙特卡洛模拟中，用于判断赎回/下修的股价序列是在风险中性测度下生成的，但该股价序列与真实测度中的股价序列存在差别，可能使赎回/下修触发时点判断的准确度降低；另外，转债的票息等现金流存在信用风险，其贴现率应当包含信用利差，而非采用无风险利率进行贴现，如前文所述，应当采用到期收益率测度进行转债定价。
- （2） 原有蒙特卡洛模拟效率较低。我们通过求未来各模拟路径中现金流贴现值的算术平均来计算转债的理论价格，即： $\hat{P}_t = \frac{1}{n} \sum_{i=1}^n P_{t,i}$ ，但平均值（一阶矩）的模拟误差相对较高，如果仅依赖增加模拟次数降低定价误差的效果有限、且效率较低，因为 $\hat{P}_t$ 的标准差 $\sigma(\hat{P}_t) = \frac{1}{\sqrt{n}} \sigma(P_{t,i})$ ，

仅靠增加模拟次数提升定价准确度会严重降低运行速度、同时存在内存不足的风险，而且定价精度提升效果与所付出的成本不成正比。

- (3) **单纯的蒙特卡洛模拟无法识别期权最优停时点。**蒙特卡洛模拟不能计算继续持有转债的预期价值，因而无法比较转债的内在价值与继续持有的价值，即难以解决持有人在转股期可能提前转股的问题。
- (4) 一部分转债可能近期已触发赎回/下修且发布了不进行赎回/下修的公告，当前正处于限制期内，此前的模型没有考虑到这一情况。
- (5) 定价程序采用的数据结构是 ndarray 数组，不支持自助求导功能，导致计算 Delta、Gamma 等指标时方法繁琐、结果偏差较大。

基于上述问题，有必要进一步完善前期的定价程序。改进后的转债定价程序代码详见附录。

(三) 转债蒙特卡洛定价模型的改进：重要性抽样

在蒙特卡洛模拟中，重要性抽样（Importance Sampling, IS）是一种通过变换测度缩小估计值方差的技术。一方面，变换测度的手段可解决风险中性测度不适用于转债定价的问题；另一方面，重要性抽样能够有效提高采样效率，无需大量增加模拟次数即可提高模拟结果的精度。

假定要估计 $E[g(X)|X \in A]$ ， X 的概率密度为 q ，则有 $E_Q[g(X)|X \in A] = \int_{x \in A} g(x)q(x)dx$ 。若 $X \in A$ 发生概率较低，可能导致蒙特卡洛模拟结果精度较低；而重要性抽样则是在 $X \in A$ 发生概率较高的测度 P 中完成抽样，再将抽样结果乘似然比（Likelihood Ratio） $\frac{q(X)}{p(X)}$ 转换到原测度 Q 中得到新结果 $Z = g(X)\frac{q(X)}{p(X)}$ ，可证明 $E_P(Z)$ 等价于 $E_Q[g(X)]$ ：

$$E_P(Z) = E_P \left[g(X) \frac{q(X)}{p(X)} \right] = \int_{x \in A} g(x) \frac{q(x)}{p(x)} p(x) dx = E_Q[g(X)]$$

其中似然比 $\frac{q(X)}{p(X)}$ 可被视作权重，调整不同样本发生的概率。在重要性抽样技术中，引入新概率密度的过程即为测度变换的过程，理论上最优的新概率密度应使得抽样方差最小。在转债定价中，测度变换是我们采用重要性抽样的首要目的，但同时重要性抽样也能达到缩小方差、提高估计精度的效果，因此采用重要性抽样技术可谓是一石二鸟。

就技术细节而言，根据前文假设，正股股价服从几何布朗运动，可推导出在真实测度下股价对数收益率满足

$$\ln \left(\frac{S_{t+1}}{S_t} \right) = \left(\mu - \frac{1}{2} \sigma^2 \right) dt + \sigma \sqrt{dt} \varepsilon_t \quad \varepsilon_t \sim N(0,1)$$

则有

$$\ln \left(\frac{S_{t+1}}{S_t} \right) \sim N \left(\left(\mu - \frac{1}{2} \sigma^2 \right) dt, \sigma^2 dt \right)$$

同理可得在到期收益率测度下

$$\ln \left(\frac{S_{t+1}}{S_t} \right) \sim N \left(\left(r - \frac{1}{2} \sigma^2 \right) dt, \sigma^2 dt \right)$$

那么真实测度即为用于取样的测度 P ，而到期收益率测度为原测度 Q 。在生成股价序列时，我们首先从正态分布 $N\left(\left(\mu - \frac{1}{2}\sigma^2\right)dt, \sigma^2dt\right)$ 中生成正股的对数收益率 x_t ，即 $\ln\left(\frac{S_{t+1}}{S_t}\right) = x_t$ ， $S_{t+1} = S_t e^{x_t dt}$ ，依此可计算出真实测度下的股价走势，用于判断是否触发赎回/下修；同时获得 x_t 在两个概率分布中对应的概率密度 $p(x_t)$ 和 $q(x_t)$ ，那么在到期收益率测度下，

$$S_{t+1} = S_t e^{x_t dt} \frac{q(x_t)}{p(x_t)}$$

从 0 时刻开始计算，则

$$S_t = S_0 \times \frac{S_1}{S_0} \times \frac{S_2}{S_1} \times \dots \times \frac{S_t}{S_{t-1}} = S_0 \times e^{x_1 dt} \frac{q(x_1)}{p(x_1)} \times e^{x_2 dt} \frac{q(x_2)}{p(x_2)} \times \dots \times e^{x_t dt} \frac{q(x_t)}{p(x_t)} = S_0 \exp\left(dt \sum_{i=1}^t x_i\right) \prod_{i=1}^t \frac{q(x_i)}{p(x_i)}$$

因此，将股价序列从真实测度转换至到期收益率测度的似然比应为每一步似然比的累乘，即

$$\text{likelihood ratio} = \prod_{i=1}^t \frac{q(x_i)}{p(x_i)}$$

这样实际上我们获得了两条股价序列：真实测度下的股价序列用于判断触发赎回/下修的时点；到期收益率测度下的股价序列用于计算转债的内在价值，并用到期收益率贴现得到当前的价值。在对转债定价时，期权价值 $\max(S - K, 0)$ 是关于股价 S 的函数，本质上也是关于股票对数收益率 x 的函数，因此我们无需真正计算出到期收益率测度下的股价序列，仅对期权价值 $\max(S - K, 0)$ 乘以似然比即可，用公式可表示为：

$$\max(S_t - K, 0) \prod_{i=1}^t \frac{q(x_i)}{p(x_i)} = \max\left[S_0 \exp\left(dt \sum_{i=1}^t x_i\right) - K, 0\right] \prod_{i=1}^t \frac{q(x_i)}{p(x_i)}$$

在代码中，为便于求导，我们将 $\max(S - K, 0)$ （ReLU 函数）替换为 ReLU 函数的平滑形式 Softplus 函数 $\text{softplus}(x) = \log(1 + e^x)$ ，则期权价值为

$$\text{softplus}\left[S_0 \exp\left(dt \sum_{i=1}^t x_i\right)\right] \times \prod_{i=1}^t \frac{q(x_i)}{p(x_i)}$$

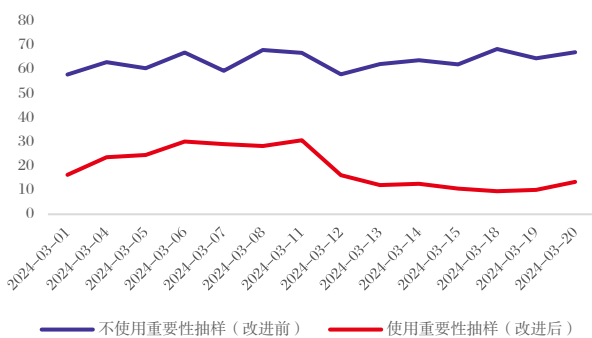
重要性抽样可同时实现缩小估计值标准差、降低定价误差的效果。我们取转债 A 在 2024 年 3 月 1 日至 3 月 20 日共 14 个交易日的数据，分别用不使用重要性抽样的原定价模型和新模型对该转债进行定价，每次定价时进行 5000 次蒙特卡洛模拟。从图 1 中可以看出，在使用重要性抽样后，估计值标准差出现显著降低，这表明重要性抽样可以实现缩减方差、提高模拟效率的效果；同时，从图 2 可以看出，不使用重要性抽样时，模型价格相对于市场价格的平均定价误差为 7.56%，而使用重要性抽样后平均定价误差下降至 2.55%，因此采用重要性抽样法也能使得模型定价更准确。

* 注：模拟价格为某一路径对应的期初转债价值，模型价格为 5000 次模拟价格的均值（即定价模型输出的定价结果）；

估计值标准差=5000 个模拟价格的标准差，数值越小表示模拟结果越集中、模拟效率越高；

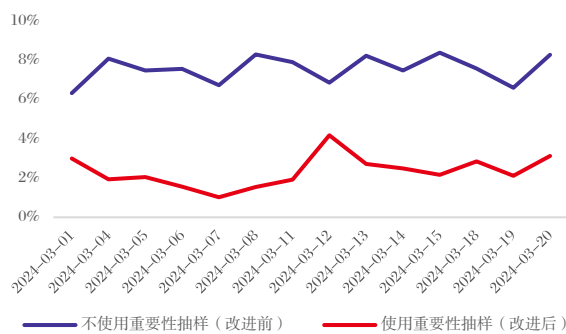
定价误差=（模型价格-市场价格）/市场价格 × 100%，数值越接近 0 表示定价越准确；下同。

图1：使用重要性抽样前后估计值标准差的对比



资料来源：Wind，中国银河证券研究院

图2：使用重要性抽样前后定价误差的对比



资料来源：Wind，中国银河证券研究院

（四）转债蒙特卡洛定价模型的改进：最小二乘蒙特卡洛模拟

转债定价面临的另一问题是进入转股期后投资者随时可转股，即具有美式看涨期权的特征。为解决这一问题，我们采用最小二乘蒙特卡洛模拟法（Least Squares Monte Carlo Simulation, LSM）。最小二乘蒙特卡洛模拟由 Longstaff & Schwartz(2001)提出，可用于美式期权定价。**LSM 能够同时契合转债路径依赖与转股期随时行权的两大特性，确定转债持有者的最佳转股时点，解决期权的最优停时问题。**

LSM 方法的基本思路是基于蒙特卡洛模拟的价格路径，对每一期的正股股价和转债的内在价值（Intrinsic Value）做最小二乘估计（OLS 回归），得到基于正股股价继续持有转债的条件期望价值，即为转债的存续价值（Continuation Value）。比较存续价值和内在价值的大小，若存续价值更高则不转股，若内在价值更高则应立即行权转股。在转债到期日，如果期权处于实值状态，则执行转股；反之不执行。

在实际操作中，我们首先按照正常的蒙特卡洛模拟生成若干条标的资产的价格路径 $S^j(t)$ ，其中 j 为路径编号；其次从转债到期/赎回时点 $t = t_K$ 开始反向回溯。对于时点 $t = t_K, t_{K-1}, \dots, t_i, \dots, t_0$ （ t_0 为当前时间），每一步用 OLS 回归法计算每条路径当前的存续价值，并且与立即行权的内在价值进行比较，从而决定当前路径该时间点的期权价值 $P^j(t_i)$ 。该方法的内涵是在同一时点上，转债价值主要受正股股价影响，即转债价值是正股股价的函数，因此我们可以构建基函数的线性组合以逼近复杂函数。拟合的线性回归方程可表示为：

$$e^{-r\Delta t}P^j(t_{i+1}) = \sum_{l=1}^L \beta_{l,i}^j f_l(S^j(t_i)) + \varepsilon^j(i)$$

其中 $P^j(t_{i+1})$ 为 t_{i+1} 时刻转债的价值， $S^j(t_i)$ 为 t_i 时刻正股的股价， $f_l(S^j(t_i))$ 表示线性组合中的基函数，这里采用第一类切比雪夫多项式，本模型中我们取 $L=2$ 。第一类切比雪夫多项式可表示为：

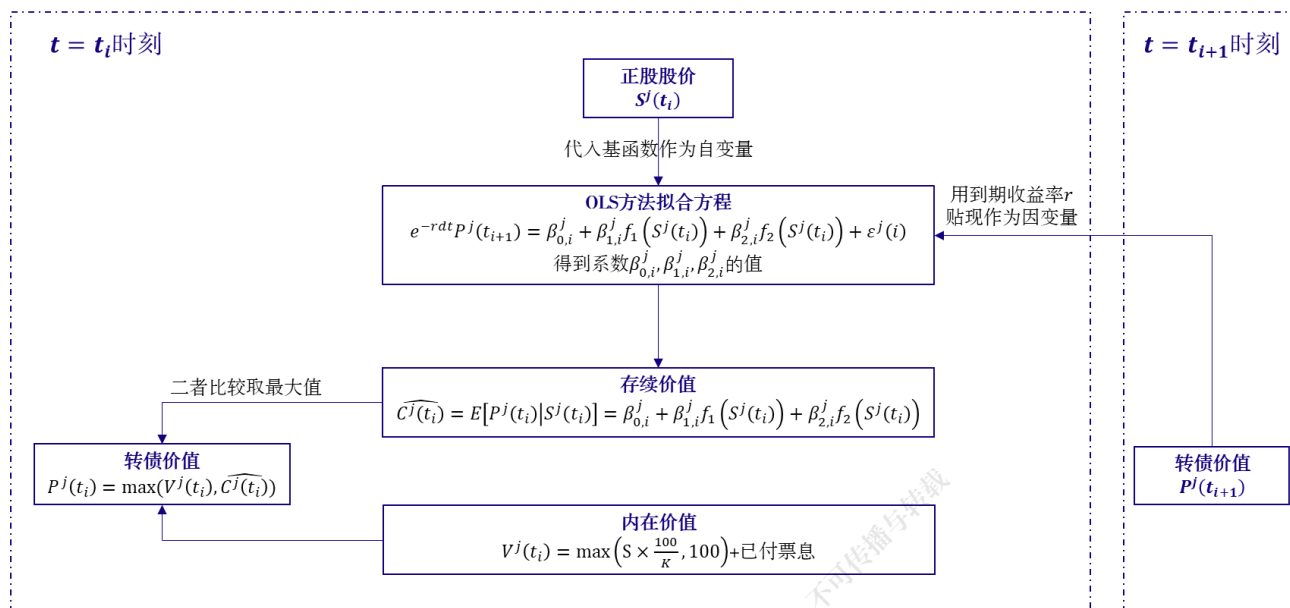
$$\begin{aligned} f_0(X) &= 1 \\ f_1(X) &= X \\ f_2(X) &= 2Xf_1(X) - f_0(X) = 2X^2 - 1 \\ f_n(X) &= 2Xf_{n-1}(X) - f_{n-2}(X) \end{aligned}$$

换言之，我们以 t_{i+1} 时刻转债价值 $P^j(t_{i+1})$ 用到期贴现率 r 贴现到 t_i 时刻的转债价值 $e^{-r\Delta t}P^j(t_{i+1})$ 作为因变量，以正股股价 $S^j(t_i)$ 对应的多项式为自变量，构建 OLS 回归来估计 t_i 时刻转债的存续价值。在通过 OLS 回归得到参数 $\beta_{l,i}^j$ 的值后， $t = t_i$ 时刻转债的存续价值 $\widehat{C^j(t_i)}$ 可表示为

$$\widehat{C^j(t_i)} = E[P^j(t_i)|S^j(t_i)] = \beta_{0,i}^j + \beta_{1,i}^j f_1(S^j(t_i)) + \beta_{2,i}^j f_2(S^j(t_i))$$

将存续价值 $\widehat{C^j}(t_i)$ 与 t_i 时刻的内在价值 $V^j(t_i)$ 进行比较，取孰高值作为 t_i 时刻转债的价值 $P^j(t_i)$ 。重复上述过程直至回溯至 t_0 时刻，即可得到路径 j 的转债价格。

图3：蒙特卡洛最小二乘法示意图



资料来源：中国银河证券研究院

（五）转债蒙特卡洛定价模型的改进：Tensor 数据结构

相比前期报告《转债新规下的定价模型更新和绝对收益策略改进》，变量的数据结构改用 **Tensor**（张量），支持 GPU 计算与自动求导，提高了运行效率，便于研究各模型参数对定价的影响。作为 PyTorch 中的基本数据类型，Tensor 是存储单一数据类型元素的高维矩阵，具有以下优势：

数据存储方面。Tensor 本质上是一个多维数组，可以存储多种数据类型以表示复杂的数据结构。Tensor 的大小是动态的，可以在运行时改变其大小。

提高程序性能方面。Tensor 存储注重内存效率，通过诸如内存重用、重叠内存分配等技术减少内存占用。此外，Tensor 运算支持并行执行，可以利用多核 CPU 和 GPU 加速计算，提高大规模数据处理与模型训练的效率。

运算方面。Tensor 支持丰富的操作，包括元素级操作和整体操作。一方面，Tensor 支持广播机制（Broadcasting），通过张量的自动扩展，允许在不同大小的 Tensor 之间进行算术运算。另一方面，Tensor 支持自动微分（Autograd）。根据 PyTorch 的官方技术文档，Autograd 是一个反向自动微分系统，其中记录了在执行操作时创建数据的所有操作。Autograd 提供了一个有向无环图，其叶节点是输入张量，根节点是输出张量。通过从根节点追溯到叶节点，程序可以通过链式法则自动计算出梯度。

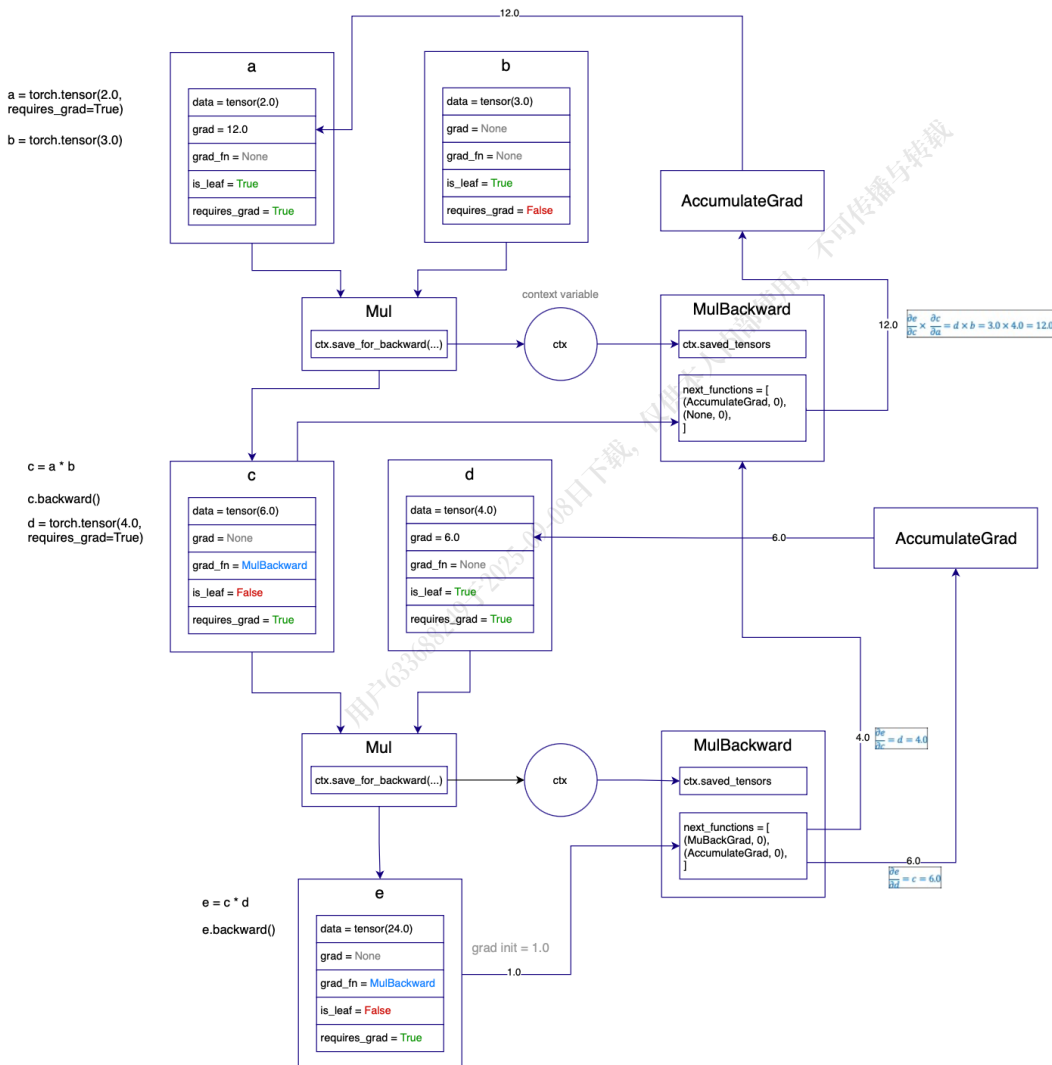
在前向计算阶段，Autograd 会执行请求的计算，并且构建一个计算图，以跟踪最终输出 Tensor 所设计到的所有操作（该图形入口为每个 torch.Tensor 的 grad_fn 属性）。随后在反向传播阶段，当调用 backward() 方法时，PyTorch 会从该节点逆向遍历计算图，使用链式法则自动计算每一个节点的梯度。

图 4 展示了 Autograd 计算 $c = a \times b, e = d \times c$ 的过程，对于 a 节点和 d 节点要求梯度计算。当创建一个 tensor 变量时，包含 data、grad、grad_fn、is_leaf、requires_grad 等关键属性，分别存储了该变量的数据域、梯度、计算图、是否为叶节点和是否需要梯度等信息。当创建变量时将 requires_grad 属性赋值为 True

请务必阅读正文最后的中国银河证券股份有限公司免责声明。

时，之后会通过反向梯度计算获得 `grad` 属性数值。在前向计算阶段，通过乘法模块计算获得 $c = a \times b$ 的数值，并将其赋给变量 `c` 的数据域，同时 `c` 的 `requires_grad` 属性继承于 `a` 的 `requires_grad` 属性，赋值为 `True`。程序对于 $e = c \times d$ 也会执行如上操作。同时，乘法模块通过上下文函数 `ctx.save_for_backward()` 创建上下文变量 `ctx`，存储反向传播阶段所需要的信息并输出给计算乘法操作反向传播函数 `MulBackward`。`MulBackward` 运算中包含 `next_functions` 属性，存储包含 `Function` 对象的元组，代表了当前操作之后执行的操作，并据此正确分配梯度。在根节点 `e` 出，程序初始化梯度为 1.0，传入 $e = d \times c$ 所对应的 `MulBackward` 运算，分别计算获得 `e` 对 `c` 和 `d` 的导数，即 $\frac{\partial e}{\partial c} = d = 4.0$, $\frac{\partial e}{\partial d} = c = 6.0$ 。前者传入 $c = a \times b$ 所对应 `MulBackward` 运算，通过链式法则获得 `e` 对 `a` 的导数，即 $\frac{\partial e}{\partial a} = \frac{\partial e}{\partial c} \times \frac{\partial c}{\partial a} = 3.0 \times 4.0 = 12.0$ ，并通过 `AccumulateGrad` 运算赋值给 `a` 的 `grad` 属性。后者传入 `AccumulateGrad` 运算，将数值存储至变量 `d` 的 `grad` 属性中。综上，实现了梯度的自动计算。

图4：Autograd 示例： $e = c \times d, c = a \times b$



资料来源：中国银河证券研究院

（六）转债定价模型参数释义

在完成了模拟方法与模型参数的一系列改进后，当前转债定价模型参数如表 1 所示。

表1：当前转债定价模型参数及解释

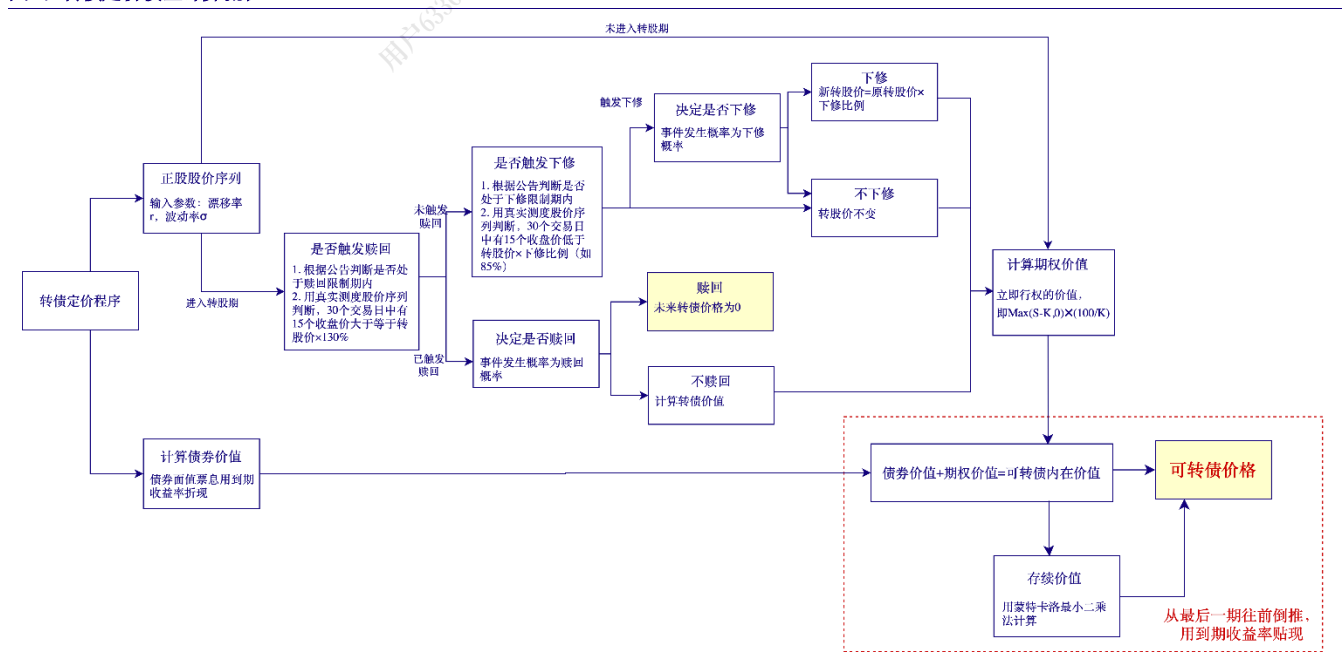
参数类别	变量名称	参数	参数解释	默认值
基础参数	rfr	贴现率	投资者可以获得的没有任何风险的收益率	0.02
	S	股票价格	股票价格	10
	K	转股价	投资者可以将可转债转换为普通股的价格	10
	payment_maturity	到期日	债券发行时确定的最终还本付息的日期	6
	simulation_rounds	模拟次数	使用蒙特卡洛方法模拟次数	500
正股参数	mue	正股收益率	可转债对应的正股的长期收益率或市场预期收益率，以年化收益率的形式表示	0.03
	sigma	正股隐含波动率	可转债对应的正股的长期波动率或市场预期波动率，以年化波动率的形式表示	0.2
利息参数	coupon	票息序列	债券在存续期间按照固定时间间隔支付给债券持有人的利息金额序列，通常以债券面值的百分比形式表示	[0.01,0.02,0.03,0.04,0.05,0.06,0.15]
	payment_year	派息年份	公司宣布分红的财政年度。在这个年度内，公司会将其盈利的一部分以股息的形式分配给股东	1
	payment_per_year	派息频率	公司支付股息的频率，即分红的时间间隔，此处设置的为年度派息	252
条款参数	redeem_prob	赎回概率	发生赎回的概率	0.15
	rdld	赎回观察的最大区间	可以进行赎回的最大时间范围	30
	redeem_rate	赎回价格	发行公司在特定条件下可以回购（赎回）可转债的价格	1.3
	rdsd	赎回观察触发区间	当观察期间内，满足标的价格大于等于赎回价格的天数大于等于该数值，则满足触发赎回条件之一	15
	relmt	不赎回的时间	在此时间区间内不发生赎回	60
	revise_prob	下修概率	发生下修的概率	0.15
	rv_long_days	下修观察最大区间	可以进行下修的最大时间范围	30
	revise_rate	下修价格	下修价格是当可转债发行公司在特定条件下决定下调可转债的转股价格时所采用的价格	0.85
	rv_short_days	下修观察触发区间	当观察期间内，满足标的价格大于等于下修价格的天数大于等于该数值，则满足触发赎回条件之一	15
	rvdrift	确定下修后的时间	在可转债发行公司决定下调可转债转股价格后，从决定生效之时开始，投资者可以行使转股权利的时间范围	20
	rvlmt	不下修时间	在此时间区间内不发生下修	90

资料来源：中国银河证券研究院

（七）正股价格路径模拟与转债定价流程

基于前文所述假设与模型改进方法，当前转债定价模型的计算流程如图 5 所示。

图5：转债定价模型计算流程



资料来源：中国银河证券研究院

该转债定价模型将可转债内在价值分为债券价值和期权价值两部分，其中债券价值可直接对转债的票息与面值贴现计算获得；而期权价值则需要采用蒙特卡洛模拟的方法，先模拟正股价格，循环判断每个交易日是否触发条款并模拟执行情况，再计算转股期权的价值。在获得每日可转债的内在价值后，再采用蒙特卡洛最小二乘法计算出当日的存续价值，二者取孰高值为转债的价值，即可转债价格的估计结果。具体而言，定价的过程可表述为：

（1）模拟每日股价序列与付息情况：根据几何布朗运动，模拟截至转债到期日正股股价时间序列，同时计算出每日重要性抽样的似然比；此时正股未来是否触发了下修或赎回的价格条件可同时获得。获取转债目前的付息情况，以 252 个交易日为间隔，模拟未来的付息时间序列。

（2）在转债存续期内，判断是否触发赎回/下修条款：本模型设定赎回与下修触发需每日进行判断，其中赎回触发在转债进入转股期后才开始进行判断；下修触发则没有进入转股期的要求，在转债存续期内每日进行判断。在判断是否触发赎回/下修条款时应同时满足两点：1）转债不能处于赎回/下修限制期内，该限制既可被模拟，也可来自实际状态（模拟前添加），互不冲突；2）正股价格在过去一段时间内满足触发赎回/下修条款的价格条件，例如正股价格在过去 30 个交易日内有 15 日低于转债价格 $\times$ 下修触发比例，则触发下修条款。

（3）若已触发赎回/下修条款，模拟是否执行：根据预设概率生成随机数，判断是否执行赎回/下修。如果执行赎回，则转债提前到期，执行赎回日转债价值等于转债前一日的内在价值，执行赎回后转债价值为 0。如果执行下修，则模拟调整转股价时间序列。假设模拟在第 i 个交易日调整转股价，新转股价则为前一交易日（ $i-1$ ）正股模拟价与第 $i-21$ 日至第 $i-2$ 日共 20 个交易日正股模拟价均值的较高者。若不执行下修/赎回，自动进入限制状态。

（4）往前倒推计算转债的内在价值与存续价值，以多条路径转债价值的平均值为转债价格：类比美式期权的定价方法，转债定价需从最后一期往前倒推。在转债没有执行赎回时，转债的内在价值等于纯债价值与转股期权价值之和；同时，以当期正股股价的切比雪夫多项式为自变量、以后一期转债价值贴现值为因变量做 OLS 回归，计算继续持有的期望价值，即为转债的存续价值。若内在价值更高，则立即行权转股；若存续价值更高，则继续持有。如此倒推至待定价日的次日，贴现至定价日并取平均值，即为当前时点转债的价格。

二、转债蒙特卡洛定价模型参数分析

（一）定价模型参数取值方法

在对可转债进行定价时，转股价、到期日、派息条款、下修与赎回条款等信息已在转债募集说明书及后续公告中有明确规定，而正股价格可直接从权益市场中获得，因此定价模型中的大部分参数都可获得一个明确的取值。在表 2 中我们列出了所有参数取值的来源。

值得注意的是，正股收益率、正股波动率、下修概率与赎回概率这 4 个参数无法从转债公告中获取，需要我们给出一个合理的估计值，下文中我们将依次讨论这 4 个参数合理的取值方法；此外，蒙特卡洛模拟的次数也直接关系到估计值的准确性与定价效率的高低，因此我们也对合理的模拟次数进行了探讨。

表2：转债定价模型参数输入值来源

参数类别	变量名称	参数	参数取值来源
基础参数	rfr	贴现率	根据转债的票息、面值、到期时间与当前价格计算 YTM
	S	股票价格	A 股市场实时价格
	K	转股价	可转债募集说明书/公告
	payment_maturity	到期日	可转债募集说明书/公告
	simulation_rounds	模拟次数	根据需要自行设置，本报告取模拟次数=2500，具体讨论过程详见后文
正股参数	mue	正股收益率	根据正股涨跌幅历史数据估计，本报告取最近 6 月年化收益率，具体讨论过程详见后文
	sigma	正股隐含波动率	根据正股涨跌幅历史数据估计，本报告取最近 6 月年化波动率，具体讨论过程详见后文
利息参数	coupon	票息序列	可转债募集说明书/公告
	payment_year	派息年份	可转债募集说明书/公告
	payment_per_year	派息频率	可转债募集说明书/公告
条款参数	redeem_prob	赎回概率	根据赎回触发与执行统计结果与 S/K 的值赋值，默认值 0.15，具体讨论过程详见后文
	rdld	赎回观察的最大区间	可转债募集说明书/公告
	redeem_rate	赎回价格	可转债募集说明书/公告
	rdsd	赎回观察触发区间	可转债募集说明书/公告
	relmt	不赎回的时间	可转债公告
	if_redeem_limited_now	当前是否赎回限制	可转债公告，取值为 0 或 1，1 表示当前处于赎回限制期内
	revise_prob	下修概率	根据下修触发与执行统计结果与 S/K 的值赋值，默认值 0.15，具体讨论过程详见后文
	rv_long_days	下修观察最大区间	可转债募集说明书/公告
	revise_rate	下修价格	可转债募集说明书/公告
	rv_short_days	下修观察触发区间	可转债募集说明书/公告
	rvdrift	确定下修后的时间	可转债募集说明书/公告
	rvlmt	不下修时间	可转债公告
	if_revise_limited_now	当前是否下修限制	可转债公告，取值为 0 或 1，1 表示当前处于下修限制期内
	limited_date	当前限制期长度	可转债公告

资料来源：中国银河证券研究院

（1）下修概率与赎回概率

下修或赎回是否被执行会受到包括正股当前股价的高低、可转债已转股的比例、公司财务状况等诸多因素影响，但本质上取决于公司管理层的决策。与正股的风险收益特征相比，下修与赎回概率一般而言相对稳定，因此我们首先探讨两个概率的合理取值。

统计自 2019 年 1 月至 2024 年 3 月全市场所有转债赎回/下修的触发与执行情况，如表 3 所示。在 1378 次下修触发事件中，有 169 次真正执行了向下修正，触发下修后执行的频率为 0.1226；在 679 次赎回触发事件中，有 95 次真正执行了提前赎回，触发赎回后执行的频率为 0.1399。考虑到近年来转债新规的发布对转债赎回和下修做了更为明确的规定，发行人执行赎回/下修的次数也有所提升，赎回和下修的概率应当比历史数据的频率统计略高，因此两个参数统一设置为 0.15。

表3：转债下修/赎回触发与执行情况

下修触发与执行请款		赎回触发与执行请款	
事件	频次	事件	频次
下修触发次数	1378	赎回触发次数	679
不向下修正	968	不提前赎回	584
向下修正	169	提前赎回	95
公告触发下修但未提议下修（默认不下修）	208		
提议下修但未出公告执行（默认不下修）	33		

资料来源：Choice，中国银河证券研究院

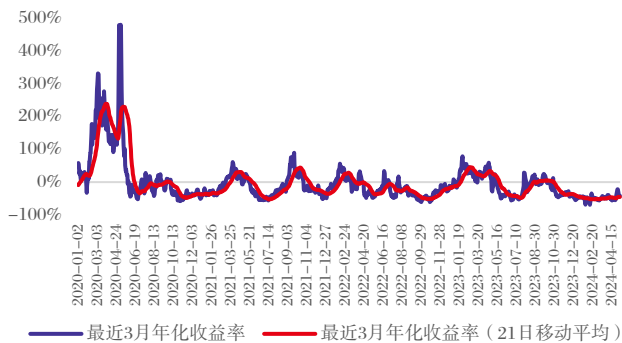
（2）正股收益率与正股波动率

估计股票收益率与波动率最常用的方法是采用最近一段时间的历史数据进行估计，那么选用多长时间的历史数据就成为讨论的重点。本报告中我们取最近 3 月、最近 6 月、最近 1 年与最近 2 年正股的日涨跌幅计算正股的年化收益率与年化波动率，代入转债定价模型中观测定价误差的大小。年化收益率与年化波动率的计算公式为：

$$\mu = \left(1 + \frac{1}{N} \sum_{i=1}^N r_i\right)^{252} - 1 \quad \sigma^2 = \frac{1}{N-1} \sum_{i=1}^N \left(r_i - \frac{1}{N} \sum_{i=1}^N r_i\right)^2$$

其中 r_i 是日涨跌幅；对于最近 3 月、6 月、1 年与 2 年，N 分别取 63、126、252 和 504。另外，由于年化收益率本身波动幅度较大，当时间区间较短时其波动更为明显，为了平滑结果、避免由于年化收益率本身波动过高而导致定价误差较大，我们对年化收益率取过去 21 日的移动平均值；而年化波动率本身波动幅度较小，不做移动平均处理。

图6：转债 A 对应正股的最近 3 月年化收益率变化



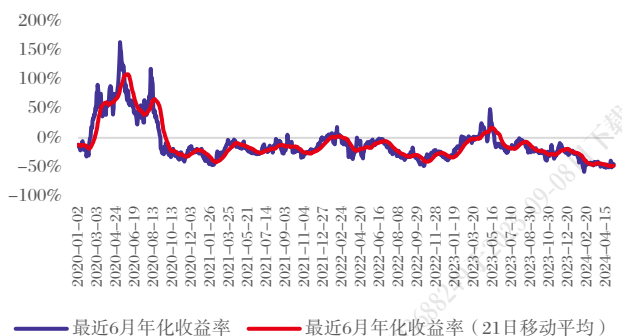
资料来源：Wind，中国银河证券研究院

图7：转债 A 对应正股的最近 3 月年化波动率变化



资料来源：Wind，中国银河证券研究院

图8：转债 A 对应正股的最近 6 月年化收益率变化



资料来源：Wind，中国银河证券研究院

图9：转债 A 对应正股的最近 6 月年化波动率变化



资料来源：Wind，中国银河证券研究院

为检验不同时间长度的历史数据的效果，我们从市场中选取正股股价 S 与转股价格 K 的比值分别在 0.5、1、1.5 左右的转债 A、B、C，分别代表处于虚值、平值与实值三种不同状态的转债，在区间 2023 年 1 月 3 日至 2024 年 3 月 22 日期间每日进行定价，并计算定价误差及其绝对值。定价时，所有转债的赎回概率和下修概率均设置为 0.15；为保证数据一致性，在输入收益率与波动率数值时，二者应当取相同的时间长度，例如最近 3 月年化收益率与最近 3 月年化波动率，模拟次数为 2500 次。

表4：不同正股收益率、波动率取值下转债平均定价误差对比

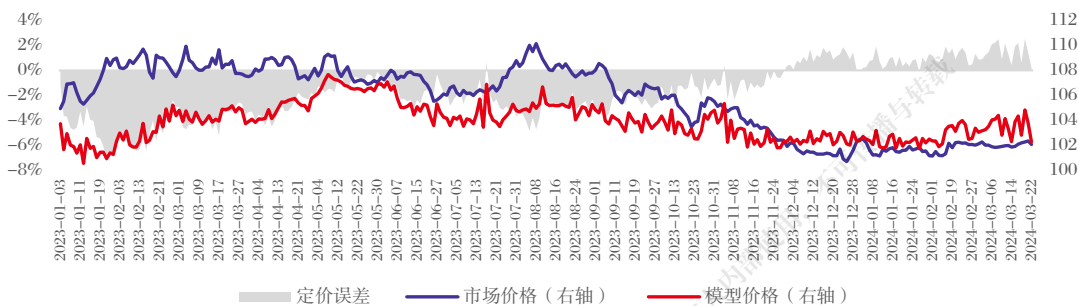
误差计算方法	转债	期权状态	平均定价误差			
			最近 3 月历史数据	最近 6 月历史数据	最近 12 月历史数据	最近 24 月历史数据
原值	转债 A	虚值	-1.97%	-1.71%	-1.42%	-0.96%
	转债 B	平值	-7.01%	-0.31%	0.72%	4.77%
	转债 C	实值	3.24%	3.83%	-7.01%	7.03%
绝对值	转债 A	虚值	2.54%	2.25%	2.18%	1.89%
	转债 B	平值	8.02%	2.73%	2.94%	4.77%
	转债 C	实值	5.12%	4.09%	8.02%	7.03%

资料来源：Wind，中国银河证券研究院

从表 4 中结果可以看出，对于转债 A 而言，4 种时间长度的定价误差相差不大，平均定价误差均在 2% 以下，误差绝对值的平均最高也仅为 2.54%。但对于转债 B 和 C 而言，采用最近 6 月历史数据都是最合适的，平均定价误差相对较低，而且误差绝对值的平均都是最低的。综合以上结果，我们认为最近 6 月年化收益率与波动率是比较合理的取值方法，这是因为转债的价格反映了投资者预期，在较短的时间区间内估计收益率与波动率更符合市场对正股走势的估计；但计算收益率与波动率的时间区间也不应过短，一方面时间过短、数据量小会使得估计结果波动过大，容易产生异常值，另一方面短期内股票预期收益率的反转效应可能更明显。在后文中，我们将采用最近 6 月年化收益率与年化波动率、赎回概率和下修概率为 0.15 的参数设置对可转债进行定价。为便于表达，后文中最近 6 月年化收益率均指 21 日滚动平均结果。

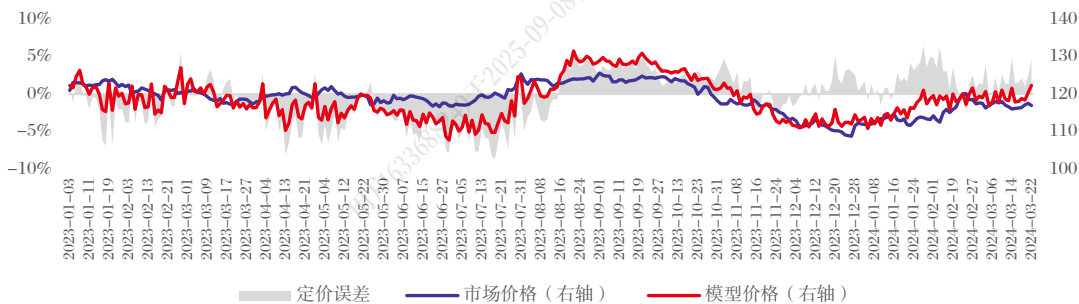
图 10、11、12 展示了转债 A、B、C 采用最近 6 月年化收益率与波动率时的定价结果与相对于市场价格的误差。总体来看，模型价格与市场价格走势一致，定价误差大部分在 $\pm 5\%$ 以内，表明蒙特卡洛模型的定价结果具有参考价值。

图10：转债 A（虚值）的定价结果与市场价格对比



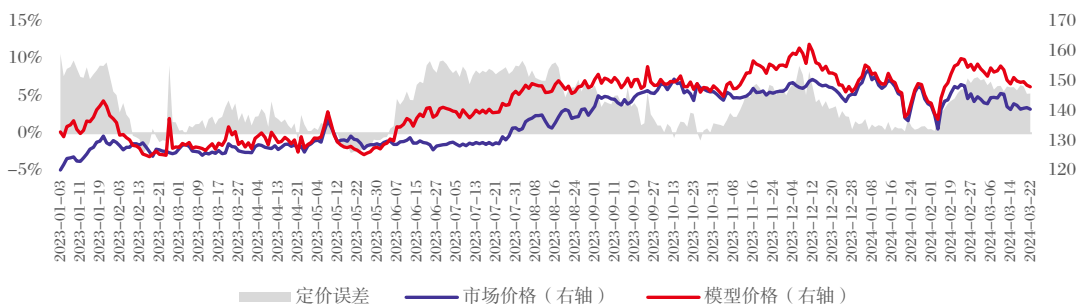
资料来源：Wind，中国银河证券研究院

图11：转债 B（平值）的定价结果与市场价格对比



资料来源：Wind，中国银河证券研究院

图12：转债 C（实值）的定价结果与市场价格对比

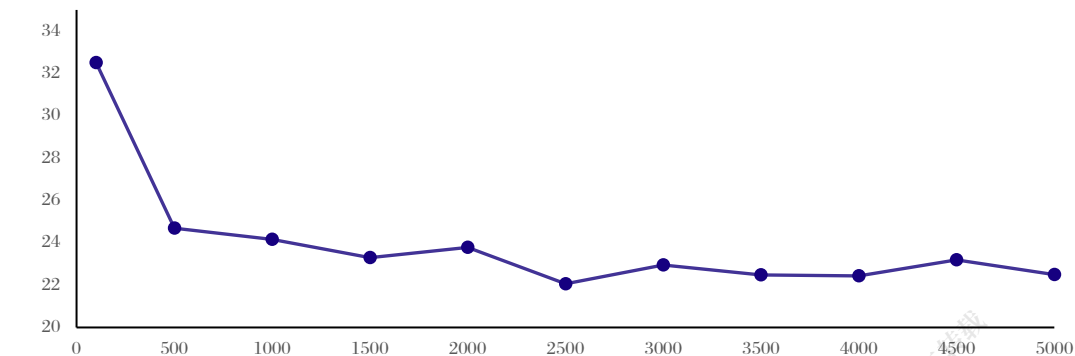


资料来源：Wind，中国银河证券研究院

（3）模拟次数

在选择合适的模拟次数时，需要兼顾估计值的准确性与定价效率。以转债 C 在 2024 年 3 月 22 日的市场数据对蒙特卡洛模拟次数进行测试，正股收益率和正股波动率分别设置为最近 6 月年化收益率和年化波动率，赎回/下修概率设置为 0.15，模拟蒙特卡洛次数为 100、500、1000、1500、2000、2500、3000、3500、4000、4500 和 5000，计算不同模拟次数下转债价格的估计值标准差。

图13：不同蒙特卡洛模拟次数下的估计值标准差



资料来源：Wind，中国银河证券研究院

从图 13 所示，从模拟次数 100 到 500，模型定价的标准差骤降，当模拟次数大于 500 时，模型定价的标准差缓慢下降，表明增加模拟次数可以提高定价准确性。在模拟次数为 2500 处平均标准差最低，且继续增加模拟次数，标准差变化不大，故本报告取蒙特卡洛模型定价次数为 2500 次。

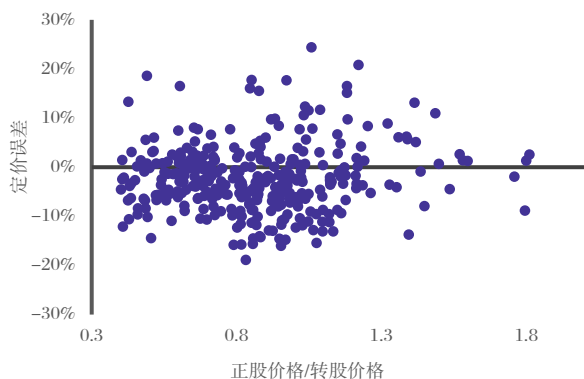
（二）模型定价准确性验证

在选定了合适的参数后，我们需要检验蒙特卡洛模型定价的准确性。考虑到转债的市场价格可能本身存在异常状况，我们首先剔除上市时间不足一年、剩余交易日不足一年的转债；其次剔除最近 6 月年化收益率高于 60%、正股价格与转股价格的比值小于 0.4 或大于 2 的转债。在计算转债价格时，我们输入的参数为最近 6 月年化收益率与波动率、赎回/下修概率为 0.15，模拟次数为 2500 次。

从时间序列的角度，前文测算正股收益率与波动率取值时，我们已经对转债 A、B、C 在时间序列上的定价结果进行了测算，在 2023 年 1 月 3 日至 2024 年 3 月 22 日这一区间上三只转债的平均定价误差分别为 -1.71%、-0.31%、3.83%，可见在时间序列上蒙特卡洛模型定价较为准确，误差处于合理范围内。

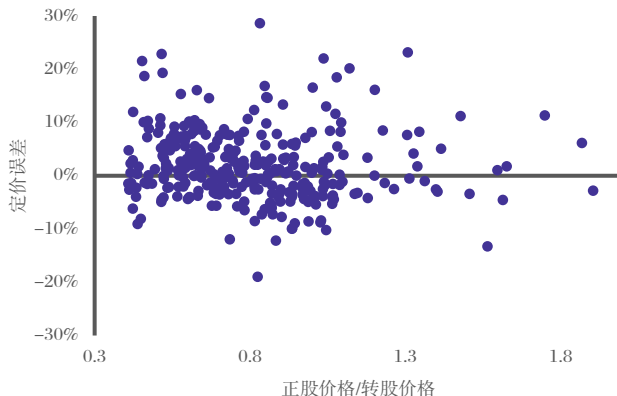
从截面的角度，我们分别取 2023 年 12 月 29 日和 2024 年 3 月 15 日全市场转债数据，用蒙特卡洛模型进行定价。2023 年 12 月 29 日，在剔除了存续时间或正股表现异常的转债后，全市场 544 只转债共剩余 349 只转债，蒙特卡洛模型定价结果的误差如图 14 所示，平均相对误差为 -2.40%。2024 年 3 月 15 日，在剔除了存续时间或正股表现异常的转债后，全市场 544 只转债共剩余 325 只转债，蒙特卡洛模型定价结果的误差如图 15 所示，平均相对误差为 1.85%。在两个时间截面上，模型平均定价误差较小，且大部分转债定价误差处于 $\pm 10\%$ 以内，可见在截面上蒙特卡洛模型对全市场转债的定价较为准确，误差处于合理范围内。由此可知，无论从个券时序，还是全市场截面角度，蒙特卡洛转债定价模型都展示出较高的准确性。

图14：2023 年 12 月 29 日全市场转债定价误差结果



资料来源：Wind，中国银河证券研究院

图15：2024 年 3 月 15 日全市场转债定价误差结果



资料来源：Wind，中国银河证券研究院

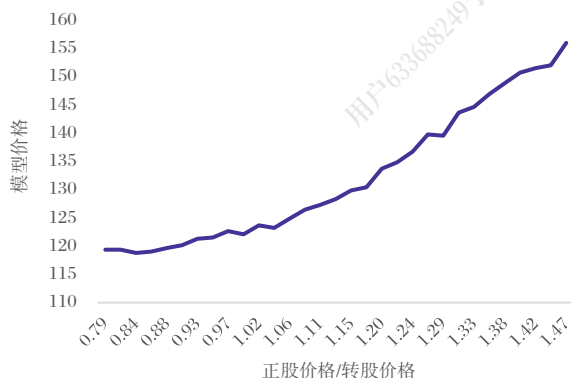
（三）定价影响因素敏感性分析

该部分主要探究正股价格、正股收益率、正股波动率与条款执行概率对可转债定价的影响。对于其他参数的取值，我们选择市场中的某一转债 D，参考其真实市场行情数据进行模拟。

（1）正股价格

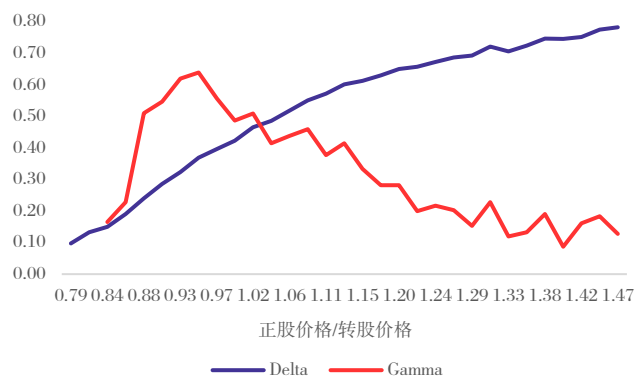
本模型中，正股价格 S 的取值是直接依照 A 股市场收盘价获得的，但实际上**正股价格是对转债价格影响最大的变量**。固定正股收益率为 5%、波动率为 30%，赎回/下修概率为 0.15，其他因素不变，令正股价格在转股价格 80-150% 的区间内变化。由图 16、17 可以看出，随着正股价格上升，转债价格呈上升趋势，变动区间为 120-155，变动幅度在 30% 左右；Delta 也呈上升趋势，在实值区间斜率放缓；Gamma 先升后降，在正股价格/转股价格接近 1 时达到最高。

图16：转债定价对正股价格的敏感性



资料来源：Wind，中国银河证券研究院

图17：Delta 和 Gamma 对正股价格的敏感性



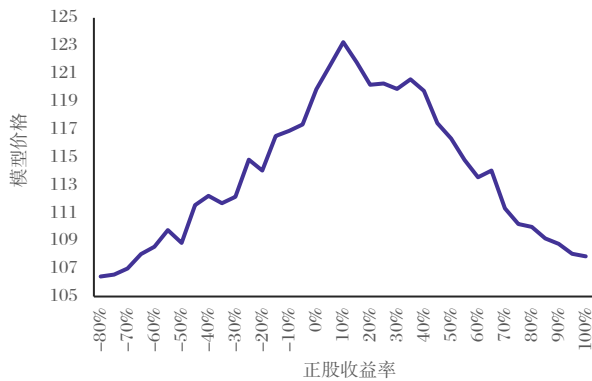
资料来源：Wind，中国银河证券研究院

（2）正股收益率

其他参数取值不变，设置正股波动率为 30%，令正股收益率在 $[-80\%, 100\%]$ 区间内取值，步长为 5%。结果如图 18、19、20 所示，当正股收益率为负时，收益率越高、转债价格越高；当正股收益率为正时，收益率越低、转债价格越低，转债价格在正股收益率取 0-5% 时达到最高值。总体而言，正股收益率对转债定价结果的影响幅度在 10% 以内；值得注意的是，这里的“价格”是蒙特卡洛模型算出的模拟价格，而

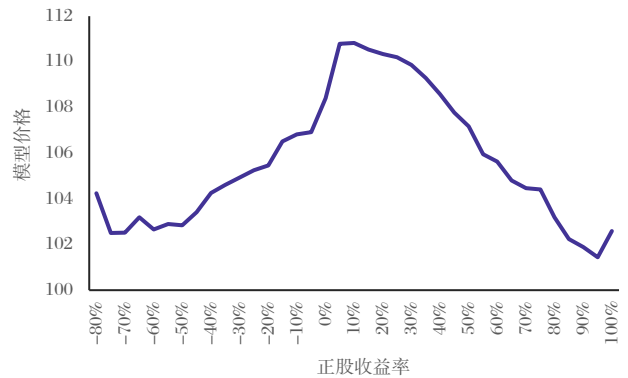
非转债的市场价格。转债价格对正股收益率的敏感性之所以呈现这样的变化，主要是由于我们在对可转债定价时进行了测度变换，将真实测度变换为到期收益率测度。

图18：正股收益率对转债定价的影响（S=K）



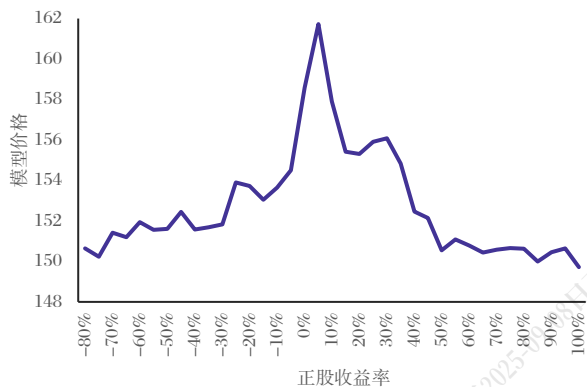
资料来源：Wind，中国银河证券研究院

图19：正股收益率对转债定价的影响（S=0.6K）



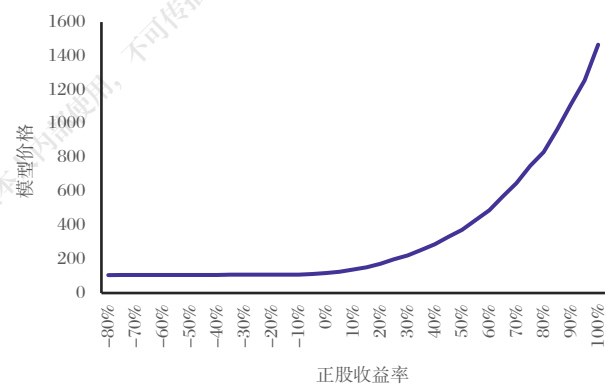
资料来源：Wind，中国银河证券研究院

图20：正股收益率对转债定价的影响（S=1.5K）



资料来源：Wind，中国银河证券研究院

图21：正股收益率对转债定价的影响（不采用重要性抽样）



资料来源：Wind，中国银河证券研究院

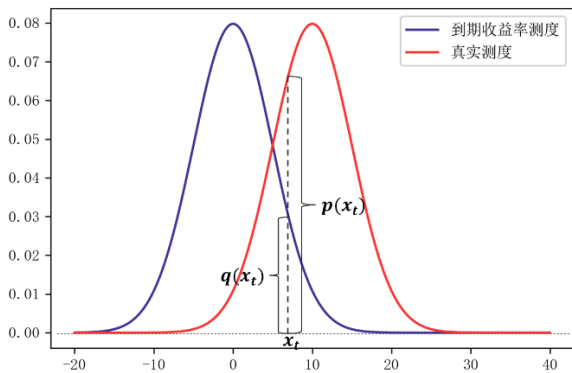
考虑到正股收益率为正时，转债价格与正股收益率之间呈现负向敏感性这一点可能有悖直觉，在此以实例证明这一结论的正确性：根据前文中对重要性抽样的论述，在到期收益率测度下，蒙特卡洛模拟生成的股价序列可表示为

$$S_{t+1} = S_t e^{x_t dt} \frac{q(x_t)}{p(x_t)}$$

其中 $q(x)$ 为到期收益率测度 $N\left(\left(r - \frac{1}{2}\sigma^2\right)dt, \sigma^2 dt\right)$ ， $p(x)$ 为真实测度 $N\left(\left(\mu - \frac{1}{2}\sigma^2\right)dt, \sigma^2 dt\right)$ ，而 x_t 是在真实测度中生成的随机样本。假设到期收益率 $r = 3\%$ ，正股收益率 $\mu = 50\%$ ，正股波动率 $\sigma = 30\%$ ，时间间隔 $dt = 1$ ，那么到期收益率测度为 $N(-0.015, 0.09)$ ，真实测度为 $N(0.455, 0.09)$ 。假设我们从真实测度中的取样结果为 $x_t = 0.455$ （即真实测度下的期望），那么此时真实测度中收益率 $e^{x_t dt} = 1.5762$ ，似然比 $\frac{q(x_t)}{p(x_t)} = 0.2931$ ，而到期收益率测度中的收益率仅为 $e^{x_t dt} \frac{q(x_t)}{p(x_t)} = 0.4620 < 1$ 。正股收益率与到期收益率差距越大，似然比就会越低，最终导致转债定价的数值下降，正股收益率为负时同理，如图 22、23 所示；只有当正股收益率与到期收益率接近时，似然比接近 1，此时转债定价结果数值较高。而我们所取的到期收益率一般在 0-5% 之间，这也是图 18、19、20 中最高点出现在 0-5% 的原因。

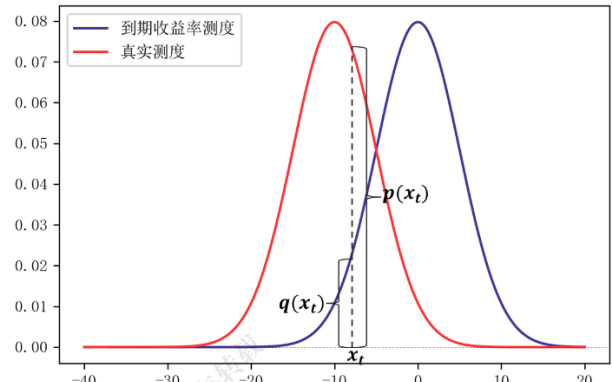
但重要性抽样仍然有存在的必要性，如果我们不做测度变化（即在计算转债价值时不乘似然比），虽然此时转债定价与正股收益率正相关，但在正股收益率较高时，转债定价也会呈现过高的异常结果，如图 21 所示，因此不管从理论还是实践结果而论，对可转债定价时都应当用重要性抽样法做测度变换。正股收益率对转债价格的影响仅仅是对定价结果的微调，后文中的正股波动率、赎回/下修概率也是如此；对定价结果起到决定性影响的仍然是正股价格 S 。

图22：重要性抽样似然比计算示例（正股收益率大于到期收益率）



资料来源：中国银河证券研究院

图23：重要性抽样似然比计算示例（正股收益率小于到期收益率）

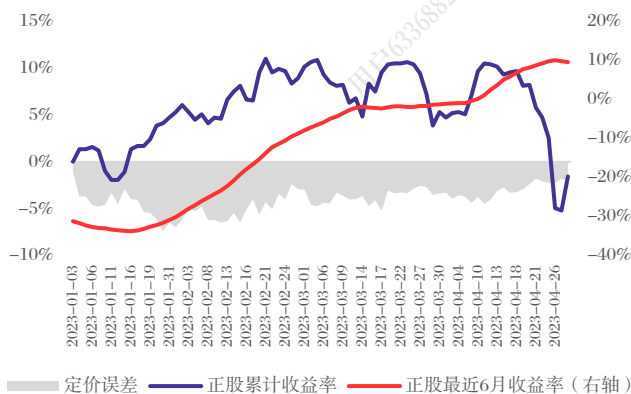


资料来源：中国银河证券研究院

回看前文中的定价结果，也可以证明这一结论。例如对于转债 A，其正股在 2023 年 1-2 月期间呈上涨趋势，而这段时间内输入的年化收益率始终为负数，这说明市场预期收益率高于我们输入的年化收益率，这导致这段时间内定价误差持续显著小于 0，因为收益率为负时转债定价呈现正敏感性。

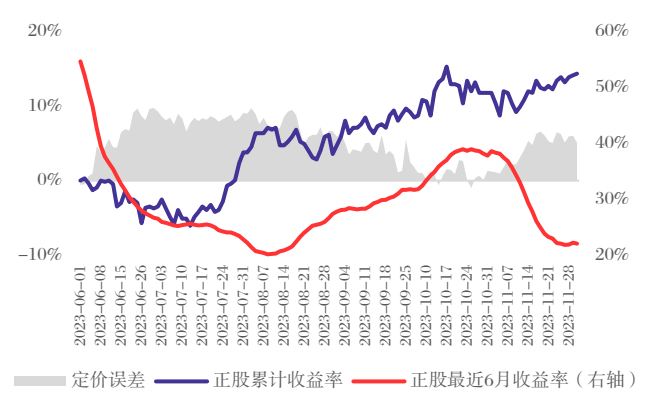
同样的，对于转债 C，在 2023 年 7 月其正股股价持续上行，表明市场对转债 C 的正股给予了一个更高的预期收益率，这段时间内我们输入的年化收益率为正数，根据收益率为正时转债定价呈现负敏感性的结论，此时定价误差持续显著大于 0 是合理的。

图24：转债 A 定价误差与正股走势、收益率的对比（2023 年 1-4 月）



资料来源：Wind，中国银河证券研究院

图25：转债 C 定价误差与正股走势、收益率的对比（2023 年 5-11 月）



资料来源：Wind，中国银河证券研究院

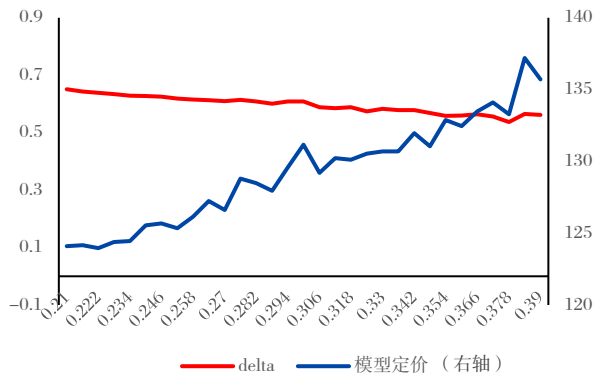
（3）正股波动率

固定正股收益率 5%、赎回/下修概率 0.15，其他参数取值不变，正股波动率在 20-40% 间取值，结果如图 26 所示。正股波动率越大，转债价格越高，当正股波动率在 20-40% 之间变动时，转债价格变动区间为 125-135 左右，正股波动率的变化对转债价格的影响幅度在 10% 以内。随着正股波动率的增大，Delta 缓慢降低。

请务必阅读正文最后的中国银河证券股份有限公司免责声明。

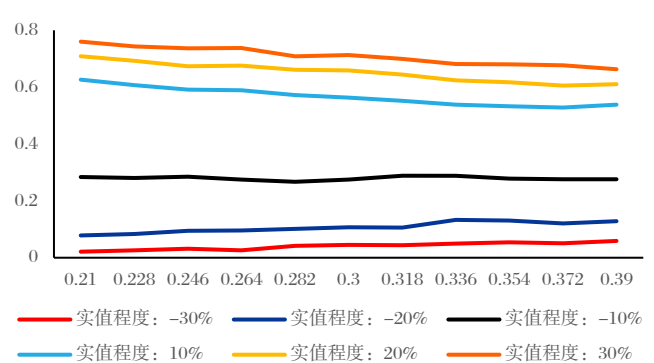
细分转债状态，改变转股价，使转债从平值状态上下浮动 30%（实值程度-30%至 30%）。在实值状态时 Delta 变化幅度较大，随正股波动率的上升 Delta 逐渐降低；在虚值状态时 Delta 变化幅度较小，随正股波动率的上升 Delta 平缓增加。

图26：正股波动率对转债定价与 Delta 的影响



资料来源：Wind，中国银河证券研究院

图27：不同实值状态下，正股波动率对转债 Delta 的影响



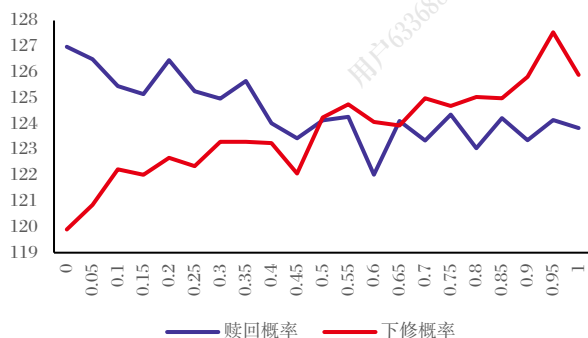
资料来源：Wind，中国银河证券研究院

（4）条款执行概率：下修概率与赎回概率

在其他因素不变的情况下，预设正股收益率为 0、波动率为 30%；初始固定下修/赎回概率为 0.5，之后分别改变赎回/下修概率，变动区间为[0,1]，间隔为 0.05，蒙特卡洛模拟次数为 2500 次。

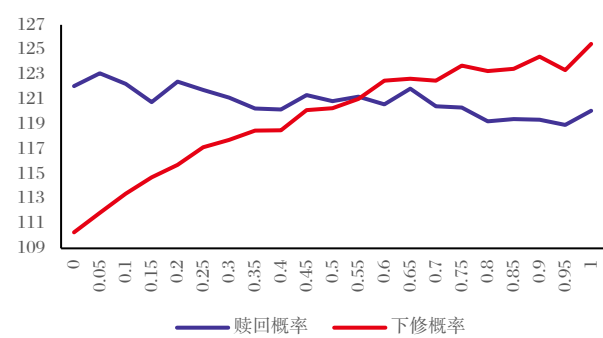
当正股价格在转股价格附近时，下修概率越高，转债价格越高；赎回概率越高，转债价格越低。这是因为如果发行人下修意愿较强，则每当正股股价下跌至触发下修时，发行人都很有可能向下修正转股价格，使得深度虚值期权变成平值期权，转债的转股价值提升，同时如果未来正股价格反弹，转债持有者也可获得更高收益，因此下修概率越高、转债价格越高。如果发行人赎回意愿较强，投资者预期转债存续时间较短，因此赎回概率越高、转债价格越低。

图28：下修与赎回概率对转债定价影响(S=K)



资料来源：Wind，中国银河证券研究院

图29：下修与赎回概率对转债定价影响(S=0.8K)



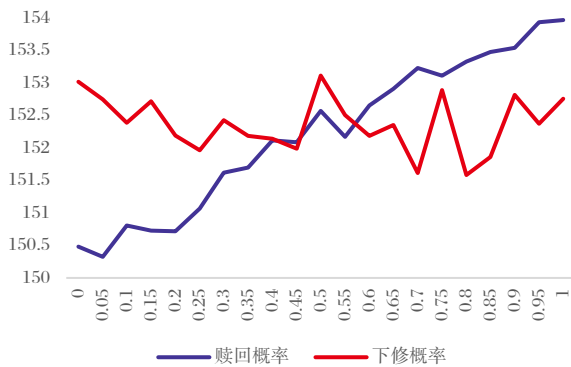
资料来源：Wind，中国银河证券研究院

当正股价格远高于转股价格时，转债价格对赎回概率的敏感性方向会发生变化。当正股价格在转股价格附近时，如果发行人赎回意愿较强，投资者预期转债存续时间较短，因此赎回概率越高、转债价格越低。但当正股价格远大于转股价格时，假设赎回概率较高，则可转债在期初即很可能发生赎回；假设赎回概率较低，则存在未来股价下跌的风险，即转债的价格也存在下跌的风险，因此赎回概率越高、转债价格越高。

期初正股股价的高低也会影响转债价格对赎回概率/下修概率的敏感性。如果当前正股股价远高于转股价格，那么股价下跌至触发下修的可能性较低，此时转债价格对下修概率的敏感性较弱；对比 S=K、

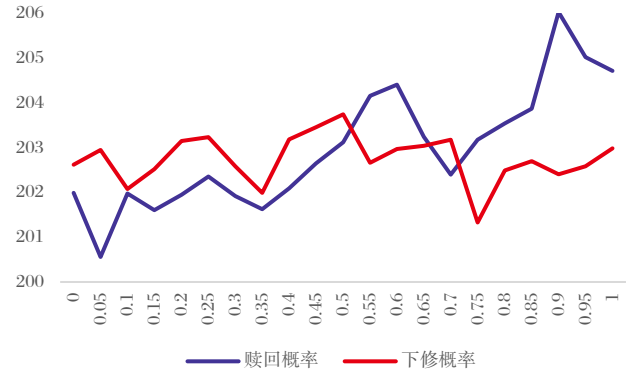
S=1.5K 和 S=2K 的结果，可发现随着正股价的升高，转债价格对下修概率变化的敏感性在逐渐降低。同理，如果当前正股价远低于转股价格，则股价上涨至触发赎回的可能性较低，此时转债价格对赎回概率的敏感性较弱。

图30：下修与赎回概率对转债定价影响(S=1.5K)



资料来源：Wind，中国银河证券研究院

图31：下修与赎回概率对转债定价影响(S=2K)



资料来源：Wind，中国银河证券研究院

基于这一特征，我们对转债的赎回/下修概率取值进行了调整。调整方法如下：当正股价格/转股价格（后文简称为 S/K）处于 0.8–1.3 之间时，赎回/下修概率仍设置为 0.15；当 $S/K < 0.6$ 时，赎回概率为 0、下修概率为 0.3；当 $S/K > 1.6$ 时，赎回概率为 0.3，下修概率为 0；剩余情况做线性插值处理。具体计算方法如表 5 所示。

表5：转债赎回/下修概率取值与 S/K 的对应关系

正股价格/转股价格 (S/K)	赎回概率	下修概率
(0,0.6]	0	0.3
(0.6,0.8]	$0.15 \times (S/K - 0.6)/0.2$	$0.3 - 0.15 \times (S/K - 0.6)/0.2$
(0.8,1.3)	0.15	0.15
[1.3,1.6)	$0.15 + 0.15 \times (S/K - 1.3)/0.3$	$0.15 - 0.15 \times (S/K - 1.3)/0.3$
[1.6,+\infty)	0.3	0

资料来源：中国银河证券研究院

用新的赎回/下修概率取值方法对深度虚值状态的转债 A、深度实值状态的转债 C 重新进行验算，结果如表 6 所示。可以看出采用了新的取值方法后，转债 A 定价误差原值的平均值从 -1.71% 缩小至 1.61%，绝对值的平均值略有增大，总体而言两种方法差别不大；转债 C 定价误差原值的平均值从 3.81% 缩小至 0.08%，绝对值的平均值从 4.09% 缩小至 2.76%，表明分段取值方法更为合理。在后文中，我们在对所有转债进行定价时，赎回/下修概率的取值均参照表 x 的方法进行分段取值；正股收益率与波动率仍采用最近 6 月的历史数据。

表6：赎回/下修概率取 0.15 和分段取值的平均定价误差对比

赎回/下修概率取值	转债 A 定价误差		转债 C 定价误差	
	原值平均值	绝对值平均值	原值平均值	绝对值平均值
0.15	-1.71%	2.25%	3.83%	4.09%
0-0.3 分段取值	1.61%	2.59%	0.08%	2.76%

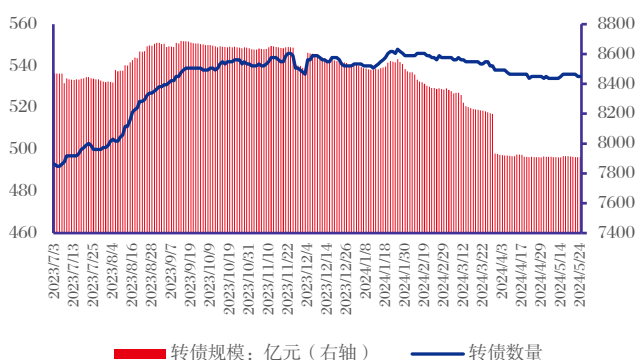
资料来源：Wind，中国银河证券研究院

三、转债市场变化

(一) 转债市场整体环境

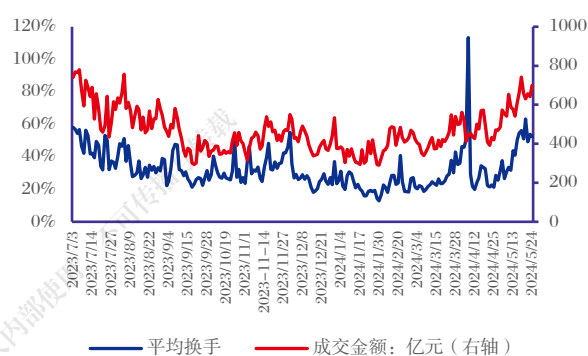
当前转债市场券源丰富，5月转债成交活跃度上升。2024Q1转债市场规模整体呈现下降趋势，其中由于中信转债（113021.SH）在2024年3月29日发生转股43.26亿股，转债规模出现明显下滑；2024Q2以来转债数量和转债规模均保持平稳，截至5月24日，转债市场现存余额7908.88亿元，比年初减少7.29%；全市场共计538只转债，比年初减少3只。自2024年2月底以来，转债市场与A股市场同步回暖，转债成交金额和平均换手均呈现上升趋势，5月以来上升趋势明显加快，5月24日转债市场成交额为698.81亿元，平均换手率为52.36%。

图32：转债市场规模变化



资料来源：Wind，中国银河证券研究院

图33：转债市场成交变化

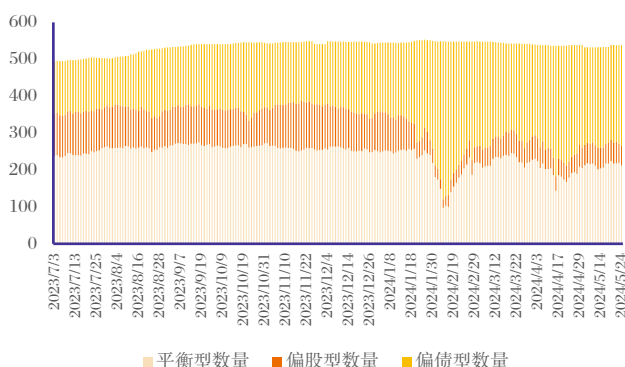


资料来源：Wind，中国银河证券研究院

当前偏股型转债数量最多，转债平均价格与转股溢价率近期下降。根据转股平价与转债价值的比值，可将转债分为偏股型转债、平衡型转债与偏债型转债，在2023年平衡型转债数量最多；在经历了2024年1月A股市场下跌后，偏股型转债数量显著减少、偏债型转债数量增加，5月以来各类型转债数量占比保持稳定，截至5月24日，偏股、平衡与偏债型转债分别有53、212、273只。自2月底A股市场回暖以来，全市场加权转股溢价率整体呈现先降后升的趋势，最近一周转债平均价格与转股溢价率同降，表明当前转债配置价值有所提升。

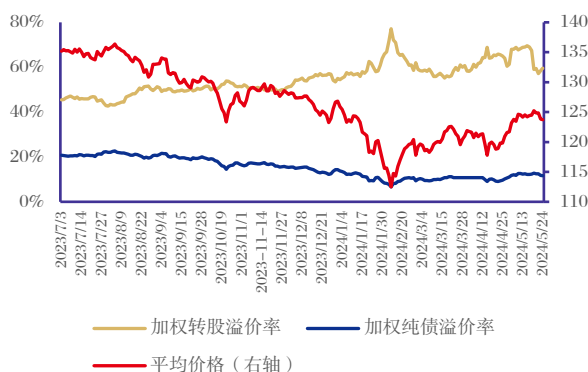
* 偏股型转债：比值大于1.2；平衡型转债：比值在0.8和1.2之间；偏债型转债：比值小于0.8。

图34：偏股型转债、平衡型转债与偏债型转债数量变化



资料来源：Wind，中国银河证券研究院

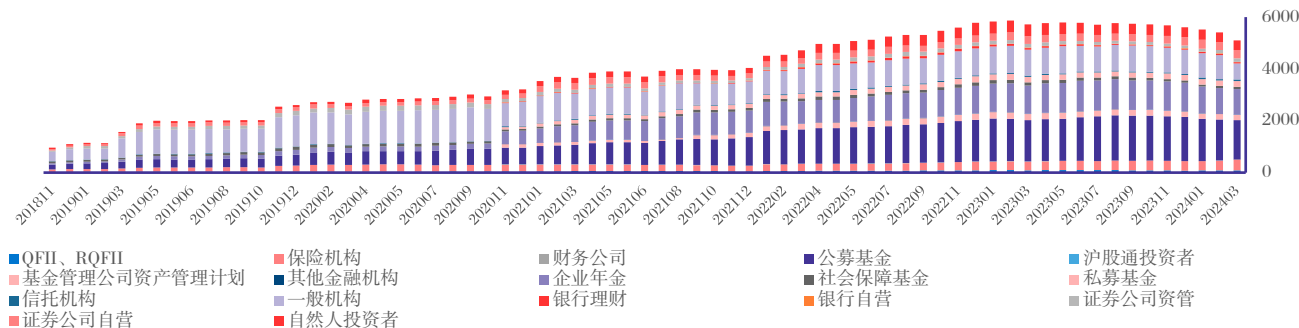
图35：转债溢价率与平均价格变化



资料来源：Wind，中国银河证券研究院

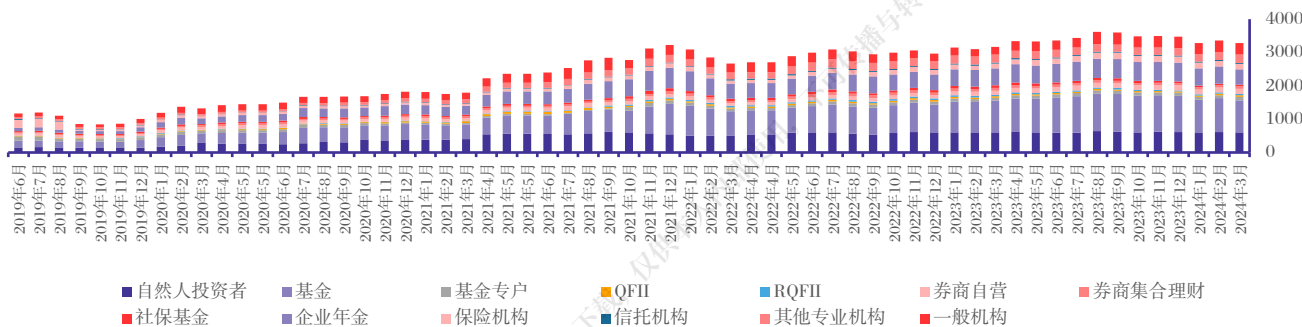
转债持有人以机构投资者为主。从 2019 年至 2024 年，机构投资者占比稳定在 90%左右。截至 2024 年 3 月，机构投资者持有转债份额约 7402.30 亿元，比 2023 年年末减少 8.49%；个人投资者（自然人投资者）持有转债份额约 976.70 亿元，比 2023 年年末减少 2.18%。

图36：上交所转债持有人变化（亿元）



资料来源：Wind，中国银河证券研究院

图37：深交所转债持有人变化（亿元）

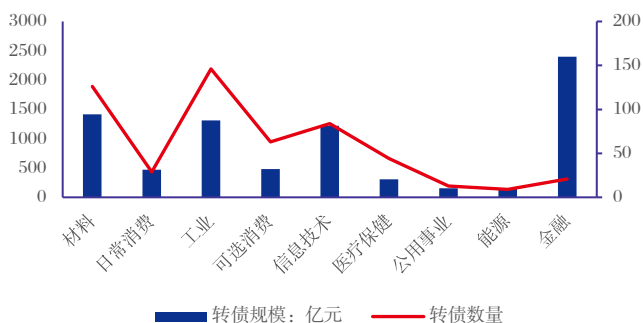


资料来源：Wind，中国银河证券研究院

（二）转债行业分布与变化

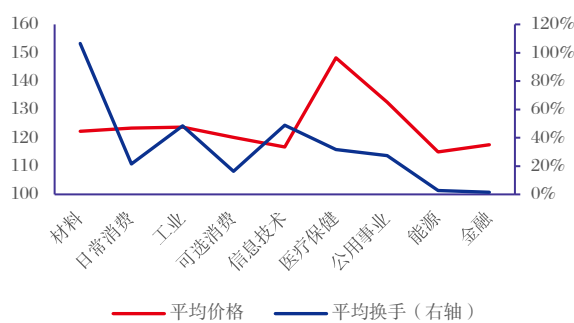
金融行业转债规模大、换手率最低；材料行业转债规模第二，换手率最高。根据 WIND 一级行业对转债进行归类，截至 2024 年 5 月 24 日，金融行业转债规模居首，余额达 2396.51 亿元，材料行业转债规模次之，余额达 1413.67 亿元；此外工业与信息技术行业转债余额也超过 1000 亿元。工业、材料行业转债数量最多，分别为 146、126 只；能源、公用事业、金融行业转债数量最少，分别为 9、13、21 只。医疗保健、公用事业行业转债平均价格最高，分别为 148.20、132.52 元；材料行业换手率最高，达 106.53%，金融、能源行业换手率最低，分别为 1.51%、2.78%。

图38：Wind 一级行业转债规模与数量分布



资料来源：Wind，中国银河证券研究院

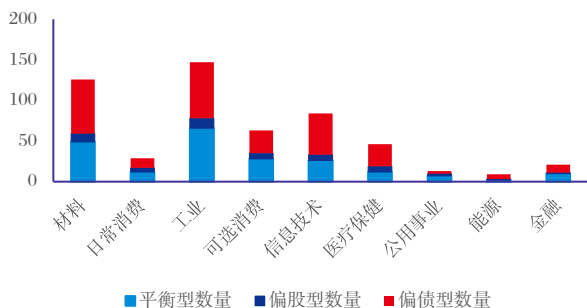
图39：Wind 一级行业转债平均价格与换手率



资料来源：Wind，中国银河证券研究院

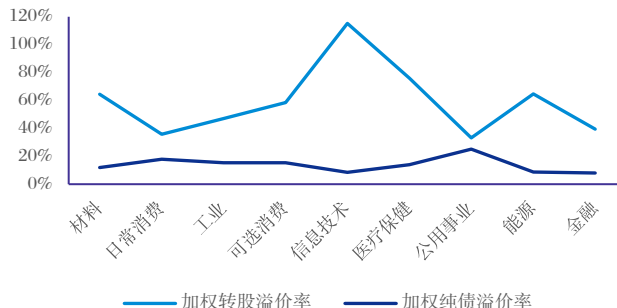
信息技术行业转债估值偏高，公用事业行业转债估值偏低。截至 2024 年 5 月 24 日，信息技术行业转股溢价率最高达 115%、纯债溢价率为 9%；同时，信息技术行业是转债市场中偏债型转债占比最高的行业（剔除能源、公用事业等转债数量较少的行业），偏债型转债有 51 只，在行业 84 只转债中占比 60.71%。公用事业行业转股溢价率最低，为 33%；纯债溢价率最高，为 25%。

图40：Wind 一级行业转债平衡、偏股与偏债型数量分布



资料来源：Wind，中国银河证券研究院

图41：Wind 一级行业转债溢价率对比



资料来源：Wind，中国银河证券研究院

四、基于蒙特卡洛定价模型的转债配置策略

当前转债市场券源丰富、交易活跃度提升，有利于可转债策略的配置，而我们的定价模型在转债策略的诸多方面均可应用。一方面，在策略构建阶段，定价模型可提供诸多有效信息，例如模型定价结果可作为交易参考、定价误差可反映市场预期变化；另一方面，在组合管理阶段，我们在定价的同时可同步计算转债的 Delta、Gamma 等敏感性系数，用于调整仓位、对冲风险等。后文中，我们将主要展示定价误差一种可能的应用方法。

（一）定价误差改进“双低”转债策略

“双低”策略是可转债经典配置策略，我们应选择价格与转股溢价率均偏低的转债，一方面价格低使得我们能够以较低价格获取转债的美式期权价值、同时债性提供下跌保护；另一方面转股溢价率低说明正股近期可能出现上涨，依据动量效应转债未来延续涨势可能性较高。但与股票低估值策略的问题一致，转债呈现“双低”特征也可能源于公司本身质地欠佳，导致转债或正股未来上涨空间有限，难以获取超额收益。因此，我们用蒙特卡洛定价模型对“双低”策略进行改进，从“双低”转债中找出市场给予乐观预期的转债进行配置。

从前文的敏感性分析中可以看出，决定转债价格高低最重要的变量是正股价格，且正股价格是直接从 A 股市场收盘价取得的，可以认为取值准确；而正股收益率、正股波动率、下修概率和赎回概率对于转债定价结果的影响相对较小、主要起到“微调”的效果，但取值来自估计的结果，可能不够准确，这构成了定价误差的来源。在这 4 个参数中，短时间内正股波动率、下修概率和赎回概率基本保持稳定，所以我们认为定价误差主要来源于正股收益率，基于定价误差可发现市场对收益率的预期差，依此配置未来市场预期乐观的可转债，构建定价误差改进“双低”择券策略。

择券策略的构建步骤如下：

第一步，构建基础样本池并用蒙特卡洛模型进行定价。基础样本池应同时满足以下条件：1）上市时间满一年，且距到期日时间大于一年；2）正股价格/转股价格在 0.4-2 之间；3）最近 6 月年化波动率低于

60%。同时，为了规避定价误差偶然性的风险，我们计算每月最后 5 个交易日的定价，并以 5 日平均定价误差作为定价误差结果。

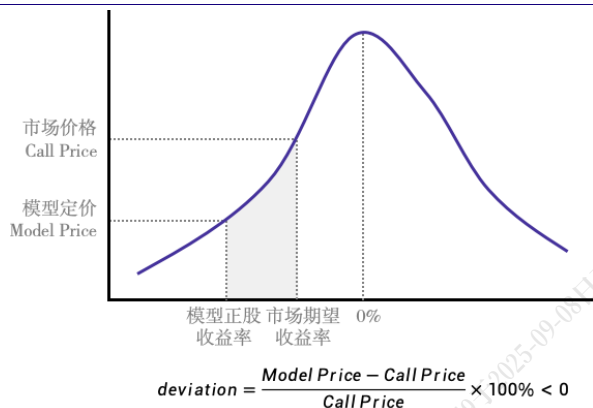
第二步，构建“双低”策略。计算“双低”指标（可转债价格+转股溢价率×100）并从小到大排序，选择样本池中指标数值最低的前 20% 转债构建“双低”策略，并作为定价误差筛选的“双低”样本池。

第三步，在“双低”策略的基础上依据定价误差构建择券策略。在前文中我们已证明了转债定价结果对正股收益率的敏感性会在到期收益率附近发生方向变化，因此我们在依据定价误差做筛选时也需要分情况讨论，具体筛选标准如下：

1）剔除“双低”样本池中定价误差绝对值>20%的转债；

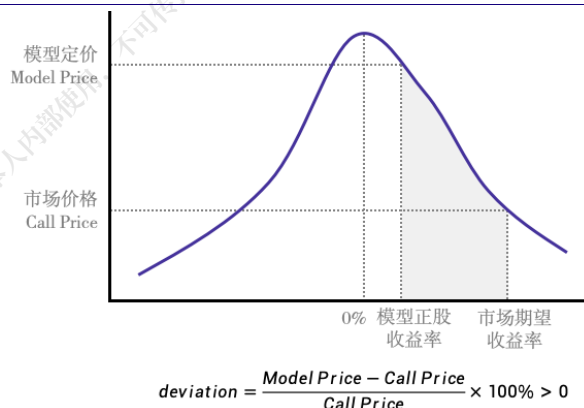
2）以沪深 300 指数代表 A 股市场表现，若当月沪深 300 收益率为正，则正股收益率整体偏高，选券时对最近 6 月正股收益率为负的转债应放松限制条件；同理，若当月沪深 300 收益率为负，则选券时对最近 6 月正股收益率为正的转债应放松限制条件。同时，若正股收益率小于 0，则应选择定价误差为负的转债；若正股收益率大于 0，则应选择定价误差为正的转债，如图 42 和 43 所示，这样的转债其市场期望收益率高于历史收益率，即我们所寻找的定价分歧点。具体择券标准详见表 7；

图42：转债配置策略示意图（正股收益率小于0）



资料来源：中国银河证券研究院

图43：转债配置策略示意图（正股收益率大于0）



资料来源：中国银河证券研究院

表7：赎回/下修概率取 0.15 和分段取值的平均定价误差对比

转债特征	市场上涨		市场下跌		定价误差 筛选条件
	正股收益率上限	正股收益率下限	正股收益率上限	正股收益率下限	
正股收益率>0	“双低”转债中正收益率的 50%分位数	0	“双低”转债中正收益率的 80%分位数	0	定价误差绝对值>2.5%且<20%
正股收益率<0	“双低”转债中负收益率的 80%分位数	无	“双低”转债中负收益率的 50%分位数	无	

资料来源：中国银河证券研究院

3）在依据正股收益率和定价误差选出的转债中，选择定价误差绝对值最高的前 20 只转债，若总数不足 20 只则不进行递补。

（二）回测结果

我们对定价误差改进“双低”策略进行回测，回测参数设置如下：

时间区间与调仓频率：2023 年 6 月 30 日至 2024 年 4 月 30 日，每月最后一个交易日调仓

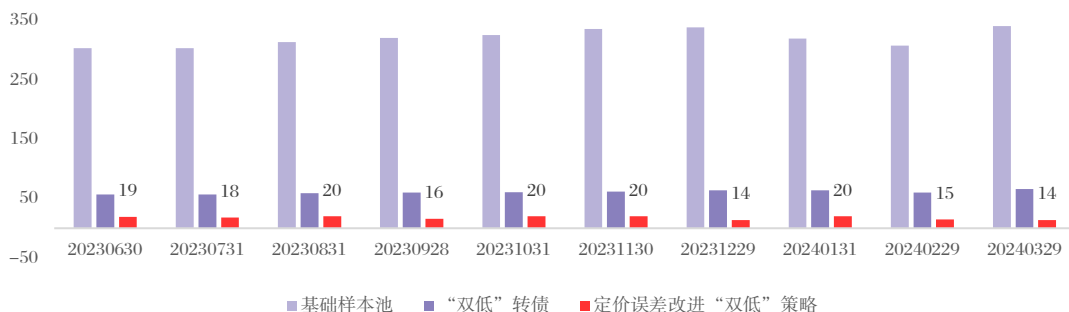
策略基准：中证转债指数（000832.CSI）

权重调整：采用等权加权和波动率倒数加权两种方法

交易费用：双边千分之三

依据前文所述策略方法，各期基础样本池、“双低”转债与策略择券数量分布如图 44 所示。

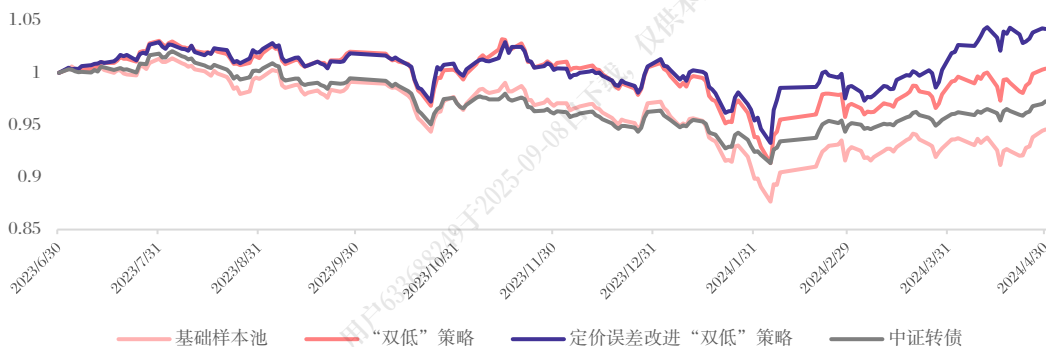
图44：各期基础样本池、“双低”转债与策略择券数量分布



资料来源：Wind，中国银河证券研究院

采用等权法对择券结果进行配置，结果如图 45 所示。从 2023 年 6 月 30 日至 2024 年 4 月 30 日，定价误差改进“双低”策略年化收益为 5.24%，相比基准中证转债指数实现超额收益 8.61%，相比基础样本池实现超额收益 11.93%，相比“双低”策略实现超额收益 4.75%；Sharpe 比率和 Calmar 比率分别为 0.5436、0.5590；最大回撤为-9.38%，相比基准和“双低”策略最大回撤均有所缩窄。

图45：等权加权定价误差改进“双低”策略净值走势对比



资料来源：Wind，中国银河证券研究院

表8：等权加权定价误差改进“双低”策略回溯业绩指标

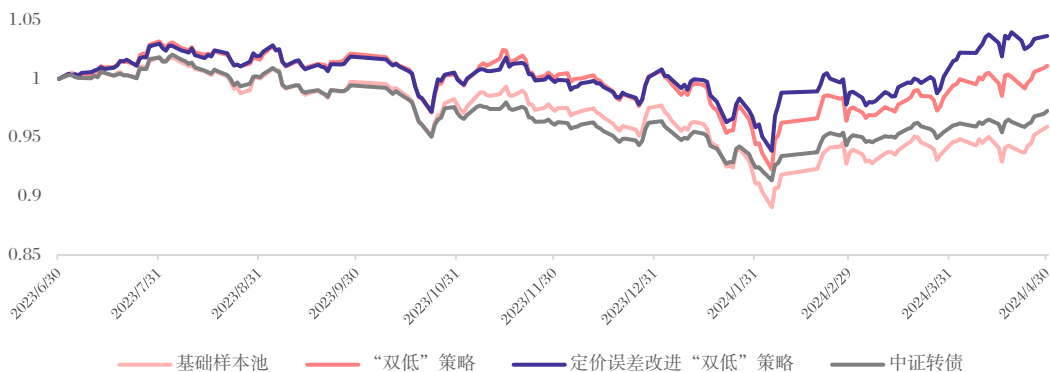
时间	年化收益率	年化波动率	夏普比率	Calmar 比率	最大回撤
基础样本池	-6.69%	9.64%	-0.6739	-0.4946	-13.54%
“双低”策略	0.49%	10.53%	0.0991	0.0427	-11.50%
定价误差改进“双低”策略	5.24%	10.44%	0.5436	0.5590	-9.38%
中证转债	-3.37%	6.64%	-0.4862	-0.3212	-10.50%

资料来源：Wind，中国银河证券研究院

采用正股波动率加权法对择券结果进行配置，结果如图 46 所示。从 2023 年 6 月 30 日至 2024 年 4 月 30 日，定价误差改进“双低”策略年化收益为 4.57%，相比基准中证转债指数实现超额收益 7.94%，相比基础样本池实现超额收益 9.58%，相比“双低”策略实现超额收益 3.18%；Sharpe 比率和 Calmar 比率分别为 0.5204、0.5154；最大回撤为-8.86%，相比基准和“双低”策略最大回撤均有所缩窄。两种配置方法均回测结果表明策略有效，无论是相比中证转债指数还是相比“双低”策略本身均可获得稳健的超额收益。

请务必阅读正文最后的中国银河证券股份有限公司免责声明。

图46：波动率倒数加权定价误差改进“双低”策略净值走势对比



资料来源：Wind，中国银河证券研究院

表9：波动率倒数加权定价误差改进“双低”策略回溯业绩指标

时间	年化收益率	年化波动率	夏普比率	Calmar 比率	最大回撤
基础样本池	-5.01%	8.80%	-0.5435	-0.3991	-12.56%
“双低”策略	1.39%	9.53%	0.1928	0.1312	-10.59%
定价误差改进“双低”策略	4.57%	9.48%	0.5204	0.5154	-8.86%
中证转债	-3.37%	6.64%	-0.4862	-0.3212	-10.50%

资料来源：Wind，中国银河证券研究院

五、结论与改进

本报告中，我们对可转债定价的理论假设重新进行了梳理，证明了可转债的定价应当在到期收益率测度下折现，并通过重要性抽样法实现了从真实测度至到期收益率测度的变换，同时实现了提高模拟效率、缩小定价误差的效果；另外，我们也采用了最小二乘蒙特卡洛法计算转债的存续价值，解决了期权的最优停时问题，并将整个定价代码的数据结构从 ndarray 数组置换为 Tensor，有效提高了定价程序的运行效率与计算精度。在完成定价模型的构建后，我们又深入讨论了模型参数的取值问题，根据测算结果，我们选择最近 6 月年化收益率的 21 天滚动均值、最近 6 月年化波动率作为正股收益率与波动率的取值，赎回/下修概率则依据正股价格/转股价格的比值在 0-0.3 之间分段取值。最后在策略构建部分我们展示了定价模型的一个可能的应用方向，通过定价误差发现市场预期收益与历史收益的分歧点，配置市场预期更乐观的转债，回溯结果证明定价误差改进“双低”策略可获得稳健的超额收益。

值得承认的是，我们的模型仍然有很多可改进的空间。例如，除正股股价与转股价格的比值以外，影响赎回/下修概率的因素还有很多，如转债是否内嵌回售条款、是否进入回售期、转债当前未转股比例等，在报告《转债新规下的定价模型更新和绝对收益策略改进》中也对此有所论述。以回售条款为例，一般而言触发回售的股价低于触发下修的股价，如果当前正股股价已经触发了下修条款，股价进一步下跌触发回售的可能性较高，一旦触发回售条款、而投资者选择执行回售，对于发行人而言资金偿付压力较大，因此当转债进入回售期或临近回售期时，发行人下修的意愿会增强。我们未来可建立合理的模型对赎回概率、下修概率等参数做出更精细的估计，使定价模型更贴合市场真实情况。

此外，在应用与实践上，本报告构建的策略仅仅是蒙特卡洛模型一个可能的应用方向，定价模型还可以广泛应用在新债上市定价、个券交易、组合管理等方面，例如计算转债的 Delta 和 Gamma 对组合做风险对冲。未来我们会进一步探索转债定价模型的应用方向，并以有效实例验证蒙特卡洛模型定价的可靠性。

六、附录

定价模型代码实例

```
class MonteCarloOptionPricing:
    """
    Main Process: MC for CB
    """

    def __init__(self, r, S0, K, T, mue, sigma,
                 div_yield=0.0, order=25, # Order of the polynomial approximations
                 simulation_rounds=10000,
                 fix_random_seed=False, use_cuda=True, **kwargs):
        """
        转债（期权基本信息）
        param S0: 当前标的价格
        param K: 转债转股价格
        param T: 剩余交易时间（年）
        param r: 贴现率（债券的到期收益率，不是无风险收益率）
        param rf: 无风险收益率
        param sigma: 波动率
        param div_yield: 分红（暂不考虑）
        param simulation_rounds: 模拟次数（高于5000次否则可能每次差异较大）
        param no_of_slices: 时间切片，一般以交易日计算
        param fix_random_seed: boolean, True or False
        """
        torch._assert(sigma >= 0, '波动率大于0')
        torch._assert(S0 >= 0, '价格大于0')
        torch._assert(T >= 0, '时间大于0')
        torch._assert(div_yield >= 0, '分红大于0')

        # Adding CUDA support
        if use_cuda:
            if torch.cuda.is_available():
                self.device = torch.device('cuda')
            else:
                self.device = torch.device('cpu')
        else:
            self.device = torch.device('cpu')

        self.S0 = S0
        self.K = K if isinstance(K, torch.Tensor) else torch.tensor(K, dtype=torch.float, device=self.device)
        self.T = T
        self.mue = torch.tensor(float(mue), requires_grad=True, device=self.device)
        self.div_yield = div_yield if isinstance(div_yield, torch.Tensor) else torch.tensor(div_yield,
                                                                                          dtype=torch.float,
                                                                                          device=self.device)

        self.no_of_slices = int(T * 252)
        self.simulation_rounds = int(simulation_rounds)

        self.rfr = r

        self.r = torch.full((self.simulation_rounds, self.no_of_slices), r / 252, dtype=torch.float).to(self.device)

        ones_matrix = torch.ones((self.simulation_rounds, self.no_of_slices), dtype=torch.float, device=self.device)
        self.sigma = ones_matrix * sigma.to(self.device)

        self.h = self.T / self.no_of_slices

        self.terminal_prices = []
        self.order = order
        self.d_t = self.T / 10

        if fix_random_seed:
            torch.manual_seed(10000)

    def stock_price_simulation(self, coupon,
                              payment_year,
                              payment_maturity,
                              payment_per_year,
                              **kwargs):
        """
        return: 现货价格及债券派息模拟
        param coupon: 转债利息序列
        param payment_year: 现阶段是派息的第N年
        param payment_maturity: 下次派息所需时间
        param payment_per_year: 每年派息次数
        """
```

```

self.exp_rf = (self.rfr - self.div_yield - (self.sigma ** 2) * 0.5) * self.h #到期收益率测度
self.exp_mu = (self.mue - self.div_yield - (self.sigma ** 2) * 0.5) * self.h #真实测度
sig = self.sigma.clone()
h_t = torch.tensor(self.h).clone()
self.exp_diffusion = sig * torch.sqrt(h_t).to(self.device)

# Use tensor operations for random number generation and price array initialization
self.z_t = torch.randn((self.simulation_rounds, self.no_of_slices)).to(self.device) #生成随机数服从标准正态分布
self.price_array = torch.zeros((self.simulation_rounds, self.no_of_slices)).to(self.device) #储存股价序列
self.price_array[:, 0] = self.S0 #第一日的股价为S0

# Vectorized calculation of price_array
diffusion_term = self.exp_diffusion * self.z_t
growth_term = self.exp_mu + diffusion_term #真实测度下生成X
self.price_array[:, 1:] = self.S0 * torch.cumprod(torch.exp(growth_term[:, :-1]), dim=1) #计算股价序列(真实测度)

# 计算似然比=fx/gx, 其中fx是目标测度(到期收益率测度), gx是生成X的测度(真实测度)
rf_t = (self.rfr - (sig ** 2) / 2) * h_t # 到期收益率测度
mu_t = (self.mue - (sig ** 2) / 2) * h_t # 真实测度
sigma_t = sig * torch.sqrt(h_t)
f = Normal(rf_t.clone(), sigma_t) # 到期收益率测度
g = Normal(mu_t.clone(), sigma_t) # 真实测度
X = mu_t + sigma_t * self.z_t
self.likelihood_ratio = torch.exp(f.log_prob(X)) / torch.exp(g.log_prob(X))
self.likelihood_ratio[:, 1:] = torch.cumprod(self.likelihood_ratio[:, :-1], dim=1) #似然比累乘

# Compute coupon_array
coupon_list = coupon.split(',')
cleaned_coupon = [float(c.strip()) for c in coupon_list]
cleaned_coupon[-1] = cleaned_coupon[-1] - 100
self.cleaned_coupon = cleaned_coupon
# cleaned_coupon[-2] = cleaned_coupon[-1] # + cleaned_coupon[-2]
# cleaned_coupon = cleaned_coupon[:-1] # Handling compensation interest

# Vectorized assignment for coupon_array
indices = torch.arange(payment_maturity - 1, self.no_of_slices, payment_per_year) #payment_per_year=252表示每年付息一次
self.indices = indices
self.coupon_array = torch.zeros((self.simulation_rounds, self.no_of_slices)).to(self.device)
self.coupon_array[:, indices] = torch.tensor(cleaned_coupon[payment_year - 1:]).to(self.device).unsqueeze(0)

# Compute terminal prices and statistics
self.terminal_prices = self.price_array[:, -1].clone()
self.stock_price_expectation = torch.mean(self.terminal_prices).to(self.device)
self.stock_price_standard_error = torch.std(self.terminal_prices).to(self.device) / torch.sqrt(
    torch.tensor(len(self.terminal_prices), dtype=torch.float)).to(self.device)

return self.stock_price_expectation, self.stock_price_standard_error

def redeem_conditions(self, pricelist, redeem_price, short_days, startpoint, redeemlimit, redeemstatus):
    """
    条件:
    1.在Long_days(30日)中, 不包括指定点 *Long_days在提取价格时使用, 函数中无需标注
    2.满足触发条件15/30
    3.未在赎回限制中(limit为1)
    4.未在赎回状态中(如果已经赎回了, 则没有必要再判断了)
    参数:
    param pricelist: 观察周期内标的的价格
    param short_days: 触发条件日期, 一般为15
    param startpoint: 起始日判断, 若近期触发条件, 则重置判断时间
    param redeemlimit: 赎回限制, 在不赎回之后, 一定时间内无法再次赎回
    param redeemstatus: 赎回状态, 若已赎回, 为1
    """
    # pricelist.clone()
    pricelist_tensor = torch.tensor(pricelist).clone().detach().requires_grad_(True).to(self.device)
    redeem_price_tensor = torch.tensor(redeem_price).clone().detach().requires_grad_(True).to(self.device)
    short_days_tensor = torch.tensor(short_days).clone().detach().to(self.device)
    startpoint_tensor = torch.tensor(startpoint).clone().detach().requires_grad_(True).to(self.device)
    redeemlimit_tensor = torch.tensor(redeemlimit).clone().detach().requires_grad_(True).to(self.device)
    redeemstatus_tensor = torch.tensor(redeemstatus).clone().detach().requires_grad_(True).to(self.device)

    conditions1 = torch.sum(pricelist_tensor.t().ge(redeem_price_tensor), dim=0).ge(short_days_tensor).long().to(
        self.device)
    conditions2 = redeemlimit_tensor.ne(1).long().to(self.device) #ne 不等于, 不在静默期内
    conditions3 = startpoint_tensor.le(0).long().to(self.device) #Le 小于等于0, 说明之前30天内未触发赎回
    conditions4 = redeemstatus_tensor.eq(0).long().to(self.device) #未发生赎回
    #四个条件同时满足则输出True
    return torch.sum(torch.stack([conditions1, conditions2, conditions3, conditions4]), dim=0) == 4
    
```

```

def revise_conditions(self, pricelist, revise_price, short_days, startpoint, reviselimit, redeemstatus):
    """
    条件:
    1. 在Long_days (30日) 中, 不包括指定点
    2. 满足触发条件15/30
    3. 未在下修限制中 (limit为-1)
    4. 未在赎回状态中 (如果已经赎回了, 则没有必要再判断了)
    参数:
    param pricelist: 观察周期内标的的价格
    param short_days: 触发条件日期, 一般为15
    param startpoint: 起始日判断, 若近期触发条件, 则重置判断时间
    param reviselimit: 下修限制, 在不下修之后, 一定时间内无法再次下修
    param redeemstatus: 赎回状态, 若已赎回, 为1
    """
    pricelist_tensor = torch.tensor(pricelist.clone()).clone().detach().requires_grad_(True).to(self.device)
    revise_price_tensor = torch.tensor(revise_price.clone()).clone().detach().requires_grad_(True).to(self.device)
    short_days_tensor = torch.tensor(short_days).clone().detach().to(self.device)
    startpoint_tensor = torch.tensor(startpoint).clone().detach().requires_grad_(True).to(self.device)
    reviselimit_tensor = torch.tensor(reviselimit).clone().detach().requires_grad_(True).to(self.device)
    redeemstatus_tensor = torch.tensor(redeemstatus).clone().detach().requires_grad_(True).to(self.device)

    conditions1 = torch.sum(pricelist_tensor.t().le(revise_price_tensor), dim=0).gt(short_days_tensor).long().to(
        self.device)
    conditions2 = reviselimit_tensor.ne(-1).long().to(self.device) #limited_status不等于-1
    conditions3 = startpoint_tensor.ge(0).long().to(self.device) #ge(0)大于等于0, 说明之前30天内没有触发过下修
    conditions4 = redeemstatus_tensor.eq(0).long().to(self.device)
    #四个条件同时满足则输出True
    return torch.sum(torch.stack([conditions1, conditions2, conditions3, conditions4]), dim=0) == 4

def initialized_cbinfo(self, rdld=30, rdsd=15, redeem_rate=1.3, redeem_prob=1, rddrift=20, rdldt=60, rvld=30,
    rvsd=15, revise_rate=0.85, revise_prob=1, rvdrt=20, rvlmt=90, convert_maturity=0):
    """
    param 参数:
    转债特定条款信息
    ----赎回----:
    赎回观察最大区间 rdld
    赎回观察触发区间 rdsd

    赎回价格 redeem_rate
    赎回概率 redeem_prob
    赎回触发后距离实施的时间 rddrift
    不赎回的时间 rdldt
    ----下修----:
    下修观察最大区间 rv_long_days
    下修观察触发区间 rv_short_days
    下修价格 revise_rate
    下修概率 revise_prob
    确定下修后的时间 rvdrt
    不下修时间 rvlmt
    ----流程----:
    转股开始时间(可开始赎回) convert_maturity
    """
    # Assign parameters to instance attributes directly
    self.rdld, self.rdsd, self.redeem_rate, self.redeem_prob, self.rddrift, self.rdldt = rdld, rdsd, redeem_rate, redeem_prob, rddrift, rdldt
    self.convert_maturity, self.rvld, self.rvsd, self.revise_rate, self.revise_prob, self.rvdrt, self.rvlmt = convert_maturity, rvld, rvsd,
    # Initialize tensors
    zeros_tensor = torch.zeros((self.simulation_rounds, self.no_of_slices), device=self.device, requires_grad=True)
    self.check_redeem = zeros_tensor.clone()
    self.limited_status = zeros_tensor.clone()
    self.start_points = zeros_tensor.clone()
    self.check_revise = zeros_tensor.clone()
    self.K_price = torch.full((self.simulation_rounds, self.no_of_slices), self.K).to(self.device)

    # Move tensors to device in a grouped manner
    tensors_to_device = [self.check_redeem, self.limited_status, self.start_points, self.K_price, self.check_revise]
    for tensor in tensors_to_device:
        tensor.to(self.device)

def ridge_regression(self, X, Y, lambda=100):
    """
    Performs a ridge regression with  $L_2$  penalty  $\lambda$ .
    That is, given a matrix  $X$  and a vector  $Y$ , it solves the least squares problem:
    
$$\hat{\beta} = \arg\min_Z ||XZ - Y||^2 + \lambda ||Z||^2$$

    Returns:  $\hat{Y} = X \hat{\beta}$ 
    """
    
```

```
I = torch.eye(X.shape[1]).to(self.device) # Create identity matrix with the same size as the number of columns in X
YY = Y.reshape(-1, 1)
X_transpose = X.transpose(1, 0)
XTX = X_transpose @ X
XTY = X_transpose @ YY

# Calculate  $\beta$  using pseudo-inverse
 $\beta$  = torch.linalg.pinv(XTX +  $\lambda$  * I) @ XTY

return torch.squeeze(X @  $\beta$ )

def polynomial_regression(self, x, y, degree, alpha=1e-6):
    """
    Performs polynomial regression using Chebyshev basis functions and ridge regression.

    Parameters:
        x: Input data (tensor)
        y: Target values (tensor)
        degree: Degree of the polynomial
        alpha: Regularization strength (default value is very small)

    Returns:
        continuation_value: Result of polynomial regression (tensor)
    """
    # Normalize the input data to the domain [-1, 1] as required by Chebyshev basis
    x_normalized = scale(x)
    # Generate Chebyshev basis functions up to the specified degree
    basis = chebyshev_basis(x_normalized, degree)
    # Perform ridge regression to estimate coefficients
    continuation_value = self.ridge_regression(basis, y, alpha)
    return continuation_value

def convertible_bond_value(self, poly_degree=2):
    """
    定价循环过程
    1 每日判断是否赎回→是否下修
    2 计算内在价值
    3 提前赎回判断

    转债特定条款信息
    ----赎回----:
    赎回观察最大区间 rdld
    赎回观察触发区间 rdsd
    赎回价格 redeem_rate
    赎回概率 redeem_prob
    赎回触发后距离实施的时间 rddrift
    不赎回的时间 rdlimt
    ----下修----:
    下修观察最大区间 rv_long_days
    下修观察触发区间 rv_short_days
    下修价格 revise_rate
    下修概率 revise_prob
    确定下修后的时间 rvdraft
    不下修时间 rvlimt
    """
    zero_tensor = torch.tensor(0, device=self.device)
    self.dis_factor = torch.zeros((self.simulation_rounds, self.no_of_slices), device=self.device,
                                   requires_grad=True)
    for i in range(self.rdld, self.no_of_slices - self.rdlimt):
        if i > self.convert_maturity: # 转债进入转股期才可能触发赎回
            rd_slice = slice(i - self.rdld, i) # 过去30个交易日作为观察期
            rd_conditions_array = self.redeem_conditions(self.price_array[:, rd_slice], rd_slice), # 使用真实价格判断是否赎回
                                self.K_price[:, i] * self.redeem_rate,
                                self.rdsd, # 赎回观察触发区间15天
                                self.start_points[:, rd_slice].sum(axis=1), # 过去30天内是否触发过赎回
                                self.limited_status[:, i], # 是否处于上次触发赎回后的静默期内
                                self.check_redeem[:, i - 1]) # check_redeem 是否已经发生赎回

            # 若触发赎回, 根据赎回概率生成0/1决定是否赎回
            redeem_judge = torch.where(rd_conditions_array,
                                       one_prob(self.redeem_prob, self.simulation_rounds).to(self.device),
                                       zero_tensor)

            # 触发赎回则当日start_points为1
            # 赎回和下修不可能同时触发, 若触发赎回, 则start_points=1; 若触发下修, start_points=-1; 其他情况等于0
            # 触发赎回后, 无论是否触发赎回, 未来一段时间内都不可能再次触发赎回 (进行赎回或静默期), 所以需要判断start_points
            self.start_points[:, i] = torch.where(rd_conditions_array, 1, 0)

            # 触发赎回而选择不赎回, 则设置未来一段时间 (rdlimt) limited_status为1, 表示静默期内不会再赎回
            self.limited_status[:, i + 1: i + self.rdlimt][(self.start_points[:, i] == 1) * (redeem_judge == 0)] = 1
            # rd_conditions_array: (N,1), redeem_judge: (N,1), limited_status: (N,M), start_points: (N,M)
            self.check_redeem[:, i + self.rddrift:][redeem_judge == 1] = 1 # 触发赎回且进行赎回, 则未来全部时间check_redeem=1
```

```

#是否触发下修无需判断是否进入转股期
rv_slice = slice(i - self.rvld, i)
rv_conditions_array = self.revise_conditions(self.price_array[:, rv_slice], #使用真实价格判断是否下修
                                             self.K_price[:, i] * self.revise_rate,
                                             self.rvld,
                                             self.start_points[:, rv_slice].sum(axis=1),
                                             self.limited_status[:, i],
                                             self.check_redeem[:, i - 1]) #已触发赎回则不会下修

#若触发下修, 根据下修概率决定是否进行下修
self.check_revise[:, i] = torch.where(rv_conditions_array,
                                     one_prob(self.revise_prob, self.simulation_rounds).to(self.device),
                                     zero_tensor)

#触发下修则当日start_points为-1
self.start_points[:, i] = torch.where(rv_conditions_array, -1, 0)
#若触发下修且尚未发生赎回, 则limited_status设为-1
self.limited_status[:, i + 1: i + self.rvld][
    (self.start_points[:, i] == -1) * (self.check_revise[:, i] == 0)] = -1
#更新下修后的转股价格
revise_K = torch.maximum(self.price_array[:, i - 1], self.price_array[:, i - 21:i - 1].mean(axis=1)).to(
    self.device).unsqueeze(dim=1) #使用真实价格改下修价 #下修价格: 最近一个交易日价格与最近20个交易日价格均值取孰高值
self.revise_K = revise_K
self.K_price[:, i + self.rvldrift:][self.check_revise[:, i] == 1] = revise_K[self.check_revise[:, i] == 1]

#计算贴现因子
rfr_tensor = torch.tensor(self.rfr, device=self.device)
self.dis_factor = torch.where(self.check_redeem == 0, torch.exp(-rfr_tensor / 252),
                              torch.tensor(1, device=self.device))

#计算票息未来现金流
future_rate = self.no_of_slices - self.check_redeem.sum(axis=1) #转债在赎回/到期前存续的天数
#票息贴现到赎回/到期日
dis_factor_forward = torch.where(self.check_redeem == 0,
                                  torch.exp(self.rfr * (future_rate.unsqueeze(1) - torch.arange(self.no_of_slices,
                                                  device=self.device)) / 252),
                                  torch.tensor(1, device=self.device))

#t=0时贴现率是exp(rT), 在赎回/到期日时贴现率是exp(r*0)=1
#相当于把coupon折现到赎回当日或债券到期当日(折现因子是单利)
self.coupon_future_array = torch.where(self.check_redeem == 0,
                                       self.coupon_array * dis_factor_forward,
                                       torch.tensor(0, device=self.device))

# Compute intrinsic value计算期权内在价值
self.intrinsic_val = 100 / self.K_price * (self.price_array - self.K_price) #期权转股价值
#计算期权价值要乘似然比, 转换为风险中性测度
#可转债内在价值=期权价值+债券价值
self.intrinsic_val = torch.nn.functional.softplus(self.intrinsic_val) * self.likelihood_ratio + 100 + torch.cumsum(
    self.coupon_future_array, axis=1).to(self.device)

self.intrinsic_val_shifted = torch.roll(self.intrinsic_val, 1, dims=-1)
self.intrinsic_val = torch.where(self.check_redeem == 1, self.intrinsic_val_shifted, self.intrinsic_val)

self.value_matrix = torch.zeros_like(self.intrinsic_val)
self.value_matrix[:, -1] = self.intrinsic_val[:, -1]

for t in range(self.no_of_slices - 2, 0, -1):
    # 最小二乘蒙特卡洛模拟(LSM), 解决期权的最优停时问题
    # continuation value即继续持有一期(不转股)的价值; intrinsic value是立即行权(转股)的价值
    continuation_value = self.polynomial_regression(self.price_array[:, t],
                                                    (self.value_matrix[:, t+1] * self.dis_factor[:, t]).to(torch.float32),
                                                    poly_degree)

    # 判断是否转股: 在转股期内, 若intrinsic value大于continuation value, 则应立即行权(转股)
    mask = torch.logical_and(self.intrinsic_val[:, t] > continuation_value,
                             torch.tensor(t > self.convert_maturity, dtype=bool))
    # 若内在价值>持有价值, 则t时刻期权价值为内在价值, 否则应为t+1时刻期权价值折现到t时刻
    self.value_matrix[:, t] = torch.where(mask, self.intrinsic_val[:, t],
                                         self.value_matrix[:, t + 1] * self.dis_factor[:, t])

# 用到期收益率dis_factor从T=1折现到T=0
self.convertible_bond_val = torch.mean(self.value_matrix[:, 1] * self.dis_factor[:, 1])
return self.convertible_bond_val

```

七、风险提示

报告结论基于历史价格信息和统计规律, 但二级市场受各种即时性政策影响易出现统计规律之外的走势, 所以报告结论有可能无法正确预测市场发展, 报告阅读者需审慎参考报告结论。证券历史收益不代表未来业绩表现, 文中观点仅供参考, 不构成投资建议。

图表目录

图 1：使用重要性抽样前后估计值标准差的对比	8
图 2：使用重要性抽样前后定价误差的对比	8
图 3：蒙特卡洛最小二乘法示意图	9
图 4：Autograd 示例： $e = c \times d, c = a \times b$	10
图 5：转债定价模型计算流程	11
图 6：转债 A 对应正股的最近 3 月年化收益率变化	14
图 7：转债 A 对应正股的最近 3 月年化波动率变化	14
图 8：转债 A 对应正股的最近 6 月年化收益率变化	14
图 9：转债 A 对应正股的最近 6 月年化波动率变化	14
图 10：转债 A（虚值）的定价结果与市场价格对比	15
图 11：转债 B（平值）的定价结果与市场价格对比	15
图 12：转债 C（实值）的定价结果与市场价格对比	15
图 13：不同蒙特卡洛模拟次数下的估计值标准差	16
图 14：2023 年 12 月 29 日全市场转债定价误差结果	17
图 15：2024 年 3 月 15 日全市场转债定价误差结果	17
图 16：转债定价对正股价格的敏感性	17
图 17：Delta 和 Gamma 对正股价格的敏感性	17
图 18：正股收益率对转债定价的影响（ $S=K$ ）	18
图 19：正股收益率对转债定价的影响（ $S=0.6K$ ）	18
图 20：正股收益率对转债定价的影响（ $S=1.5K$ ）	18
图 21：正股收益率对转债定价的影响（不采用重要性抽样）	18
图 22：重要性抽样似然比计算示例（正股收益率大于到期收益率）	19
图 23：重要性抽样似然比计算示例（正股收益率小于到期收益率）	19
图 24：转债 A 定价误差与正股走势、收益率的对比（2023 年 1-4 月）	19
图 25：转债 C 定价误差与正股走势、收益率的对比（2023 年 5-11 月）	19
图 26：正股波动率对转债定价与 Delta 的影响	20
图 27：不同实值状态下，正股波动率对转债 Delta 的影响	20
图 28：下修与赎回概率对转债定价影响（ $S=K$ ）	20
图 29：下修与赎回概率对转债定价影响（ $S=0.8K$ ）	20
图 30：下修与赎回概率对转债定价影响（ $S=1.5K$ ）	21
图 31：下修与赎回概率对转债定价影响（ $S=2K$ ）	21
图 32：转债市场规模变化	22
图 33：转债市场成交变化	22
图 34：偏股型转债、平衡型转债与偏债型转债数量变化	22
图 35：转债溢价率与平均价格变化	22
图 36：上交所转债持有人变化（亿元）	23
图 37：深交所转债持有人变化（亿元）	23
图 38：Wind 一级行业转债规模与数量分布	23
图 39：Wind 一级行业转债平均价格与换手率	23
图 40：Wind 一级行业转债平衡、偏股与偏债型数量分布	24
图 41：Wind 一级行业转债溢价率对比	24
图 42：转债配置策略示意图（正股收益率小于 0）	25
图 43：转债配置策略示意图（正股收益率大于 0）	25

图 44：各期基础样本池、“双低”转债与策略择券数量分布	26
图 45：等权加权定价误差改进“双低”策略净值走势对比	26
图 46：波动率倒数加权定价误差改进“双低”策略净值走势对比	27

表格目录

表 1：当前转债定价模型参数及解释	11
表 2：转债定价模型参数输入值来源	13
表 3：转债下修/赎回触发与执行情况	13
表 4：不同正股收益率、波动率取值下转债平均定价误差对比	14
表 5：转债赎回/下修概率取值与 S/K 的对应关系	21
表 6：赎回/下修概率取 0.15 和分段取值的平均定价误差对比	21
表 7：赎回/下修概率取 0.15 和分段取值的平均定价误差对比	25
表 8：等权加权定价误差改进“双低”策略回测业绩指标	26
表 9：波动率倒数加权定价误差改进“双低”策略回测业绩指标	27

分析师承诺及简介

本人承诺以勤勉的执业态度，独立、客观地出具本报告，本报告清晰准确地反映本人的研究观点。本人薪酬的任何部分过去不曾与、现在不与、未来也将不会与本报告的具体推荐或观点直接或间接相关。

马普凡 金融工程首席分析师。十二年量化研究经验，曾任职于华泰柏瑞基金、广发证券、中信证券，2022 年加入银河证券研究院。

免责声明

本报告由中国银河证券股份有限公司（以下简称银河证券）向其客户提供。银河证券无需因接收人收到本报告而视其为客户。若您并非银河证券客户中的专业投资者，为保证服务质量、控制投资风险、应首先联系银河证券机构销售部门或客户经理，完成投资者适当性匹配，并充分了解该项服务的性质、特点、使用的注意事项以及若不当使用可能带来的风险或损失。

本报告所载的全部内容只提供给客户做参考之用，并不构成对客户的投资咨询建议，并非作为买卖、认购证券或其它金融工具的邀请或保证。客户不应单纯依靠本报告而取代自我独立判断。银河证券认为本报告资料来源是可靠的，所载内容及观点客观公正，但不担保其准确性或完整性。本报告所载内容反映的是银河证券在最初发表本报告日期当日的判断，银河证券可发出其它与本报告所载内容不一致或有不同结论的报告，但银河证券没有义务和责任去及时更新本报告涉及的内容并通知客户。银河证券不对因客户使用本报告而导致的损失负任何责任。

本报告可能附带其它网站的地址或超级链接，对于可能涉及的银河证券网站以外的地址或超级链接，银河证券不对其内容负责。链接网站的内容不构成本报告的任何部分，客户需自行承担浏览这些网站的费用或风险。

银河证券在法律允许的情况下可参与、投资或持有本报告涉及的证券或进行证券交易，或向本报告涉及的公司提供或争取提供包括投资银行业务在内的服务或业务支持。银河证券可能与本报告涉及的公司之间存在业务关系，并无需事先或在获得业务关系后通知客户。

银河证券已具备中国证监会批复的证券投资咨询业务资格。除非另有说明，所有本报告的版权属于银河证券。未经银河证券书面授权许可，任何机构或个人不得以任何形式转发、转载、翻版或传播本报告。特提醒公众投资者慎重使用未经授权刊载或者转发的本公司证券研究报告。

本报告版权归银河证券所有并保留最终解释权。

评级标准

评级标准	评级	说明
评级标准为报告发布日后的 6 到 12 个月行业指数（或公司股价）相对市场表现，其中：A 股市场以沪深 300 指数为基准，新三板市场以三板成指（针对协议转让标的）或三板做市指数（针对做市转让标的）为基准，北交所市场以北证 50 指数为基准，香港市场以摩根士丹利中国指数为基准。	行业评级	推荐：相对基准指数涨幅 10%以上
		中性：相对基准指数涨幅在-5% ~ 10%之间
		回避：相对基准指数跌幅 5%以上
	公司评级	推荐：相对基准指数涨幅 20%以上
		谨慎推荐：相对基准指数涨幅在 5% ~ 20%之间
		中性：相对基准指数涨幅在-5% ~ 5%之间
		回避：相对基准指数跌幅 5%以上

联系

中国银河证券股份有限公司 研究院

深圳市福田区金田路 3088 号中洲大厦 20 层

上海浦东新区富城路 99 号震旦大厦 31 层

北京市丰台区西营街 8 号院 1 号楼青海金融大厦

公司网址：www.chinastock.com.cn

机构请致电：

深广地区：程 曦 0755-83471683 chengxi_yj@chinastock.com.cn
 苏一耘 0755-83479312 suyiyun_yj@chinastock.com.cn
 上海地区：陆韵如 021-60387901 luyunru_yj@chinastock.com.cn
 李洋洋 021-20252671 liyangyang_yj@chinastock.com.cn
 北京地区：田 薇 010-80927721 tianwei@chinastock.com.cn
 褚 颖 010-80927755 chuying_yj@chinastock.com.cn